

CSE450 Capstone Project Report

ProcureNext: An Intelligent and Auditable E-Tendering Platform for Private-Sector Procurement in Bangladesh

Submitted by

Tanvin Islam Abir	2105020
Md. Mahmud Hasan	2105027
Tawhid Umar	2105028
Fatin Sadab	2105029
Safayat Saif	2105030

Supervised by

Khaled Mahmud Shahriar
Assistant Professor, Department of CSE, BUET

Mohammad Sadat Hossain
Lecturer, Department of CSE, BUET

Industry Partner

Advanced Information Systems (AllTender.com)
External Mentor: Md Annur Rahman



Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

Session 2025–2026
29 September 2026

CANDIDATES' DECLARATION

We declare that the work presented in this report, titled “ProcureNext: An Intelligent and Auditable E-Tendering Platform for Private-Sector Procurement in Bangladesh”, is the outcome of the engineering work carried out by us under the supervision named on the title page.

All prior work, third-party code, libraries, data, copyrighted material, and tools used in the project have been properly acknowledged. Any use of generative AI tools has been disclosed (Section 6.5.3). We accept accountability and responsibility for the accuracy and integrity of this report and for our individual contributions to the project.

Tanvin Islam Abir (2105020)

Md. Mahmud Hasan (2105027)

Tawhid Umar (2105028)

Fatin Sadab (2105029)

Safayat Saif (2105030)

ACKNOWLEDGEMENT

We are deeply grateful to our supervisors, Khaled Mahmud Shahriar, Assistant Professor, and Mohammad Sadat Hossain, Lecturer, Department of Computer Science and Engineering, BUET, for their guidance throughout the project. Their feedback on requirement engineering, on the choice between a monolithic and a microservice architecture, and on the evaluation of the intelligent components shaped many of the decisions described in this report.

We thank our industry partner, Advanced Information Systems, the company behind All-Tender.com, and in particular our external mentor Md Annur Rahman, who shared practical knowledge of how private-sector tendering is run in Bangladesh and reviewed our requirements. His feedback kept the platform grounded in real practice.

We also acknowledge the Bangladesh Public Procurement Authority, whose public e-GP portal served as the reference point for our requirement analysis, and the maintainers of the open-source projects on which ProcureNext is built.

ABSTRACT

Public procurement in Bangladesh has been digitised through the national e-GP portal, which by November 2025 had processed over one million tenders for more than 14,000 procuring entities. Private-sector procurement has no equivalent. Companies still circulate tender notices through newspapers and e-mail, collect bids in physical drop boxes where stronger local parties can intimidate competitors, print thousands of pages per tender, and keep no verifiable record of who did what. Existing private tender portals are bulletin boards: they list notices, but offer no bidding, no evaluation support, and no audit trail.

ProcureNext is a multi-tenant web platform that carries a private-sector tender from draft to award and contract completion. Organisations register once as buyer, vendor, or both; their members work under role-based permissions set by an organisation owner. Buyers can import an e-GP-format tender notice PDF, and the platform extracts its structured fields automatically. They publish tenders publicly or only to enlisted vendors, receive bids with supporting documents, compare them side by side, and obtain an explainable, AI-assisted evaluation that combines financial scoring, document compliance, semantic relevance and an LLM rubric. Vendors discover tenders through hybrid keyword-plus-semantic search. Tender-scoped group chats are encrypted with AES-256-GCM. Every critical action is written to a SHA-256 hash-chained audit log with Merkle-root archives, and a usage-based token economy funds the platform.

The system is built as a FastAPI modular monolith with a stateless Python ML service, PostgreSQL with pgvector, Celery and Redis, and a Next.js front end. It is deployed with Docker on AWS EC2 through a CI-gated GitHub Actions pipeline, and 344 automated backend tests pass. In experiments, the parser recovered 27 of 30 fields exactly from three real e-GP notices. On a 25-query retrieval benchmark, keyword search returned nothing for 12 queries, while the deployed hybrid search ranked a relevant tender first for all 25 (MRR 1.00 against 0.52), at a median end-to-end latency of 8 ms.

Keywords: e-procurement; e-tendering; semantic search; hybrid retrieval; tamper-evident audit log; role-based access control

Contents

Candidates' Declaration	i
Acknowledgement	ii
Abstract	iii
1 Problem Identification and Formulation	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Complexity of the Problem	3
1.4 Project Objectives and Scope	5
2 Preliminary Research	7
2.1 Literature Review	7
2.1.1 E-procurement and its outcomes	7
2.1.2 Lexical, dense and hybrid retrieval	7
2.1.3 Document understanding and LLM-assisted evaluation	8
2.1.4 Explainable recommendation	8
2.1.5 Secure, auditable transaction systems	8
2.2 Existing Solutions and Technologies	9
2.3 Market and User Research	11
2.4 Feasibility Study	11
3 Requirements Analysis	13
3.1 Project Objectives and Constraints	13
3.1.1 Stakeholders	13

3.1.2	Functional requirements	13
3.1.3	Non-functional requirements	15
3.1.4	Constraints	15
3.2	Regulatory Requirements and Standards	15
3.3	Formulation of Solutions	19
4	System Design and Implementation	21
4.1	Functional Design and Sub-problem Partitioning	21
4.1.1	Sub-problems and modules	22
4.1.2	How standards and safety shaped the design	23
4.2	Design Refinement and Simulation	24
4.2.1	Use cases and data flow	24
4.2.2	Data model	26
4.2.3	Tender lifecycle	26
4.2.4	Interaction design	26
4.2.5	Algorithms	28
4.2.6	Design calculations	30
4.2.7	Design refinements	30
4.3	Application of Tools	31
4.4	Implementation	32
4.4.1	Scale of the implementation	32
4.4.2	Key modules	35
4.4.3	Development process	35
4.5	Deployment	36
5	Investigation and Evaluation	38
5.1	Experimental Investigation	38
5.1.1	E1: Tender-notice parsing (RQ1)	38
5.1.2	E2: Hybrid semantic search vs. keyword search (RQ2)	39

5.1.3	E3: Similarity ceiling for off-topic queries (RQ3)	41
5.1.4	E4: Latency on CPU (RQ4)	41
5.1.5	E5: Financial scoring behaviour (RQ5)	41
5.2	Performance Evaluation	42
5.2.1	Automated verification	42
5.2.2	Evaluation against objectives	42
5.2.3	Evaluation against functional requirements	42
5.2.4	Non-functional requirements	45
5.3	Deviations and Design Revisions	46
6	Project Management, Teamwork, and Impact	47
6.1	Project Management and Finance	47
6.1.1	Planning and Risk Management	47
6.1.2	Budgeting and Resource Identification	47
6.1.3	Economic Analysis and Financial Prospecting	49
6.1.4	Sustainability in Business and Commercialisation	50
6.2	Individual and Teamwork	51
6.2.1	Participation and Contribution	51
6.2.2	Collaboration and Conflict Resolution	52
6.2.3	Leadership and Direction	53
6.2.4	Multidisciplinary Engagement	53
6.3	Impact on Society	54
6.4	Environmental Impact and Life Cycle Sustainability	55
6.5	Ethics	56
6.5.1	Equity and Inclusivity	56
6.5.2	Accountability and Personal Responsibility	56
6.5.3	Proper Use of Intellectual Property	56
6.5.4	Professionalism and Ethical Codes	58

7 Conclusion	59
7.1 Conclusion and Future Work	59
7.2 Continuous Engagement and Self-directed Learning	60
References	62
A Functional Requirements with Acceptance Criteria	66
B Selected API Endpoints	69

List of Figures

4.1	ProcureNext system architecture as deployed. Solid arrows are synchronous calls; the dashed LLM call is optional, with a deterministic fallback.	21
4.2	Use-case diagram (actors: unregistered user, registered user, buyer, seller, payment gateway, OTP provider)	24
4.3	Level-0 (context) data-flow diagram	25
4.4	Level-1 data-flow diagram	25
4.5	Entity–relationship diagram of the core schema	27
4.6	Tender state machine	27
4.7	Collaboration diagram for bid submission	28
4.8	Public landing page	33
4.9	Buyer dashboard: tender portfolio with status filters, token balance and search .	33
4.10	Tender creation with one-click import from an e-GP notice PDF, packaged lots, key dates and required specification documents	34
4.11	Vendor bid submission with financial amount, proposal and buyer-mandated documents	34
5.1	E2: MRR by system and query type	40
6.1	Project schedule as executed (February–September 2026)	48

List of Tables

1.1	Complex engineering problem characteristics (P1–P7) and evidence from this project	4
1.2	Knowledge profiles referenced by P1	5
2.1	Comparison of existing solutions with ProcureNext	10
3.2	Functional requirements (baseline of 24 February 2026)	13
3.1	Stakeholders, expectations and conflicts	16
3.3	Non-functional requirements and measurable targets	17
3.4	Regulatory requirements and standards	17
3.5	Weighted evaluation of candidate solutions (scores 1–5)	20
4.1	Module partitioning	22
4.2	Tools and technologies: justification and limitations	31
5.1	E1: field extraction from real e-GP notices (deterministic parser)	39
5.2	E2: retrieval quality on the 25-query benchmark	40
5.3	E3: semantic matches returned for off-topic queries (ceiling $\delta < 0.8$)	41
5.4	E4: measured latency (20 repetitions, CPU only)	41
5.5	E5: financial scoring and outlier flag on a worked example	42
5.6	Automated test suite by area (344 passed, 2 skipped live-database tests, 0 failed)	43
5.7	Achievement of project objectives	43
5.8	Status of functional requirements	44
6.1	Major risks and mitigation	48
6.2	Itemised monthly production budget (BDCCL catalogue)	49
6.3	Monthly break-even and revenue scenarios (6 bids per tender)	50

6.4	Roles and contributions of team members	51
6.5	Societal impacts and design responses	54
6.6	Negative environmental impacts and mitigation	55
6.7	Third-party software, models and services	57

Chapter 1

Problem Identification and Formulation

1.1 Background and Motivation

Procurement, meaning the purchase of goods, works and services through a competitive process, is one of the largest flows of money in any economy. In Bangladesh, public procurement alone amounts to about US\$30 billion a year [1]. Since 2011 the government has run this spending through the electronic Government Procurement (e-GP) portal, `eprocure.gov.bd`, operated by the Bangladesh Public Procurement Authority (BPPA, formerly CPTU) [2, 3]. By November 2025 the portal had crossed one million tenders. It served 143,374 registered tenderers and 14,175 procuring entities, and invited 20,000 tenders in a single month [1]. It publishes Annual Procurement Plans, live tenders, awarded contracts and a debarment list to the public, and has become an international reference for e-GP adoption in a developing economy [4].

Private-sector procurement has been left behind. Garment manufacturers, pharmaceutical companies, real-estate developers, hospitals, universities and NGOs also buy at tender scale, but the e-GP system is built around the Public Procurement Act 2006 and its Rules [5, 6] and is not open to private buyers. Our requirement analysis, the review of existing portals, and discussions with our industry mentor at Advanced Information Systems (the company behind AllTender.com) point to a common private-sector workflow:

- **Circulation by newspaper and e-mail.** Buyers pay for newspaper advertisements and e-mail PDF notices to vendors they already know. Vendors outside that circle rarely hear about the opportunity.
- **Physical submission.** Sealed bids are dropped into a tender box at the buyer's premises. This creates what practitioners call the "*muscle power*" problem: a strong local party can physically prevent competitors from reaching the box and so win uncontested.
- **Paper and travel.** A single tender cycle can generate thousands of printed pages, including copies of trade licences, TIN and VAT certificates, technical proposals and bank guarantees, together with repeated travel for purchase of documents, pre-bid meetings and submission.
- **No record.** Evaluation happens in spreadsheets and e-mail threads. When a bidder disputes an award, there is no trustworthy record of when bids arrived, who opened them, or what

changed.

Online private tender sites exist, such as `bdtender.com`, `tenderbazar.com` and `AllTender.com`, but they are essentially *bulletin boards*. They aggregate notices, often scraped from newspapers, and let vendors browse them. Submission, evaluation and award still happen offline (Section 2.2).

The people affected are therefore (i) private buyers, who pay high advertising and handling costs and receive fewer, less competitive bids; (ii) vendors, especially small and medium enterprises (SMEs) outside Dhaka, who cannot see or safely reach opportunities; and (iii) society at large, which bears the cost of collusion, paper waste and unaccountable spending. Empirical work on e-procurement shows that moving tendering online raises competition and quality and reduces opportunities for collusion [7, 8]. The e-GP portal has shown that the approach works at national scale in Bangladesh. The motivation for ProcureNext is to bring that proven approach to the private sector, adding the flexibility private buyers need and the intelligence (semantic search, automated parsing, evaluation assistance) that the government portal lacks.

1.2 Problem Statement

Problem. Design, build and deploy a secure, multi-tenant web platform that carries a private-sector tender through its full lifecycle, from notice to award to contract completion, so that (a) any verified vendor can discover and bid on eligible tenders remotely; (b) buyers keep control over visibility and evaluation; (c) every critical action is recorded in a tamper-evident way; and (d) the platform reduces manual effort through document understanding, semantic retrieval and explainable evaluation support, while keeping humans responsible for every award decision.

Entities. Let $\mathcal{O}$ be the set of organisations, each with a type in $\{Buyer, Vendor, Both\}$ and a verification state in $\{Pending, Verified, Rejected\}$. Let $\mathcal{U}$ be the set of users; each user u belongs to at most one organisation with a role $r(u) \in \{Owner, ProcurementOfficer, Finance, Viewer\}$. A tender $t \in \mathcal{T}$ has a buyer $b(t) \in \mathcal{O}$, text (title, description, eligibility), budget bounds $[\beta_{min}, \beta_{max}]$, a deadline $d(t)$, a visibility $v(t) \in \{Public, Restricted\}$, a set of required documents $R(t)$, optional lots, and a state in $\{Draft, Published, Closed, Awarded, Cancelled\}$. A bid $x \in \mathcal{B}$ links a vendor organisation to a tender with a financial amount $a(x)$, a textual proposal and a set of uploaded documents $D(x)$.

Inputs and outputs. The inputs are tender forms or tender-notice PDFs, bids with documents, search queries, organisation registration documents (NID, trade licence, TIN, VAT), payments for platform tokens, and messages. The outputs are tender listings and details filtered by the viewer's rights, ranked search results, ranked vendor recommendations with reasons, per-bid evaluation scores with justifications, awards, contracts, notifications, a token ledger, and an audit trail.

Properties to guarantee.

- (G1) *Access control*. A user may perform an action on a resource only if the resource's organisation, visibility and the user's role allow it. A *Restricted* tender is visible only to its buyer and to vendors enlisted by that buyer.
- (G2) *Lifecycle integrity*. State transitions follow the tender state machine: bids are accepted only while t is *Published* and $\text{now} < d(t)$, and exactly one bid is accepted per award.
- (G3) *Financial consistency*. Token debits and credits are atomic with the actions that cause them, and every balance change appears in an append-only ledger.
- (G4) *Tamper evidence*. Any modification, deletion or reordering of a past audit record is detectable.
- (G5) *Confidentiality*. Private messages and sensitive documents are readable only by authorised parties.

Quantities to optimise. The platform should maximise retrieval quality (Recall@ k and mean reciprocal rank of relevant tenders for a vendor's query), the accuracy of fields extracted from tender PDFs, and the usefulness of evaluation scores. It should do so while keeping interactive latency low (search well under one second) and running within the cost of a single commodity server (Section 2.4).

Assumptions. Tender documents are mostly in English and text-based (e-GP notices are generated PDFs). Organisation documents are verified by a human platform administrator. Contract signing remains a physical, legal act under the Contract Act 1872, so the platform records its confirmation rather than executing it. Payment gateway settlement is outside the platform's trust boundary.

Knowledge required. The problem draws on database systems (relational modelling, transactions and isolation, full-text indexing), information retrieval and natural-language processing (dense sentence embeddings, approximate nearest-neighbour search, rank fusion), computer security (authentication, RBAC, authenticated encryption, cryptographic hash chains and Merkle trees), distributed and asynchronous systems (task queues, idempotent workers), software engineering (modular architecture, automated testing, CI/CD), human-computer interaction, and domain knowledge of procurement regulation and practice.

1.3 Complexity of the Problem

A complex engineering problem must have characteristic P1 and some or all of P2–P7 (Table 1.1). P1 requires in-depth knowledge at the level of one or more of the knowledge profiles K3–K8 listed in Table 1.2. ProcureNext exhibits all seven.

Table 1.1: Complex engineering problem characteristics (P1–P7) and evidence from this project

Code	Definition	Evidence from this project
P1	Depth of knowledge required. Cannot be resolved without in-depth engineering knowledge at the level of one or more of K3, K4, K5, K6, or K8, which allows a fundamentals-based, first-principles analytical approach.	K3: relational modelling of 47 tables and ACID transactions for token debits and bid acceptance; cosine geometry of embeddings. K4: dense retrieval, HNSW/pgvector indexing, reciprocal rank fusion, SHA-256 hash chaining and Merkle roots, AES-GCM. K5: architecture trade-off (monolith vs. microservices), asynchronous pipeline design. K6: Docker, CI/CD, cloud deployment. K8: SBERT/MiniLM, RRF, tamper-evident logging and LLM-as-judge literature (Section 2.1).
P2	Range of conflicting requirements. Involves wide-ranging or conflicting technical, engineering, and other issues.	Transparency vs. confidentiality (public listings vs. restricted tenders and sealed bids); intelligence vs. data privacy (LLMs vs. keeping embeddings local); rich features vs. single-server budget; automation vs. human accountability for awards; openness to SMEs vs. strict KYC verification.
P3	Depth of analysis required. Has no obvious solution and requires abstract thinking and originality in analysis to formulate suitable models.	No off-the-shelf model scores bids. We formulated a four-component composite with a piecewise financial function and median-based outlier detection (Section 4.2). Keyword search alone fails on vocabulary mismatch, and pure vector search loses exact matches, so we designed a thresholded hybrid with rank fusion.
P4	Familiarity of issues. Involves infrequently encountered issues.	Parsing Bangladesh e-GP notice layouts; “muscle power” interference in physical bid submission; multi-organisation group chats in which both companies’ owners must be present; bid-bond and token economics.
P5	Extent of applicable codes. Is outside problems encompassed by standards and codes of practice for professional engineering.	The PPA 2006/PPR 2008 govern public, not private, procurement; there is no Bangladeshi standard for private e-tendering, for auditing AI-assisted bid evaluation, or for tamper-evident logs. We adapted e-GP practice, OWASP and ISO/IEC guidance (Section 3.2).
P6	Extent of stakeholder involvement and conflicting requirements. Involves diverse groups of stakeholders with widely varying needs.	Buyers, vendors (large firms and SMEs), organisation owners vs. members, the platform administrator, the industry partner, payment providers and regulators; e.g. buyers want maximal vendor information while vendors want their pricing sealed from competitors (Table 3.1).
P7	Interdependence. Is a high-level problem including many component parts or sub-problems.	Twelve mounted backend modules, a separate ML service, asynchronous workers and a 27-page front end depend on one another: publishing a tender touches RBAC, tokens, the ML embedding pipeline, notifications and the audit log in one flow (Section 4.1).

Table 1.2: Knowledge profiles referenced by P1

Code	Description
K3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.
K4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
K5	Knowledge that supports engineering design in a practice area.
K6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.
K8	Engagement with selected knowledge in the research literature of the discipline.

1.4 Project Objectives and Scope

Objectives

The project's objectives are the following verifiable goals. Chapter 5 evaluates the system against each one.

- O1. Multi-tenant identity and access.** Organisations register with KYC documents that a platform administrator verifies; owners invite members by e-mail and assign one of four roles; every protected endpoint enforces RBAC and tender visibility (FR-04–FR-06, FR-12, FR-21).
- O2. Complete tender lifecycle.** Buyers draft, publish, edit, cancel and delete tenders, including single-item and multi-lot packages with required-document lists. Expired tenders close automatically (FR-08).
- O3. Remote, structured bidding and award.** Vendors submit, update and withdraw bids with documents before the deadline. Buyers compare bids side by side, award one, and complete the resulting contract with mutual reviews (FR-10, FR-11, FR-13, FR-14, FR-16, FR-17).
- O4. Intelligent assistance.** (a) Extract structured fields from e-GP-format tender PDFs; (b) provide hybrid semantic search that outperforms keyword search on vocabulary-mismatched queries; (c) recommend vendors with explanations; (d) score bids with an explainable multi-component evaluation (FR-03, FR-09, FR-10, FR-15).
- O5. Communication.** Provide encrypted intra-company and tender-scoped inter-company messaging with real-time in-app notifications (FR-07).
- O6. Accountability.** Log every state-changing request in an append-only, hash-chained audit log whose integrity can be verified on demand (FR-22).

- O7. Sustainable economics.** Implement a token (credit) economy with admin-configurable pricing and packages and an append-only transaction ledger (FR-18, FR-20).
- O8. Engineering quality.** Reach an automated test suite covering every mounted module, CI on every change, and CI-gated containerised deployment to a cloud host.

Scope

Included: the public portal (about, policies, legal, news, events, help, active public tenders); organisation and user onboarding; the buyer, vendor and administrator dashboards; the tender, bid, award, contract and review workflows; the ML service (PDF parsing, embeddings, bid scoring); messaging and notifications; the token economy; the audit subsystem; CI/CD and deployment.

Excluded: legally binding digital signature of contracts (contracts are signed physically and confirmed in the system, per FR-13); live settlement through a licensed payment gateway, since the purchase flow runs in sandbox mode until a merchant account is in place; automated NID/TIN verification against government APIs (FR-26), which is replaced by manual administrator review; dispute management and exportable analytics (FR-23, FR-24), which the requirements already marked as tentative; native mobile applications; and Bangla-language document processing.

Chapter 2

Preliminary Research

2.1 Literature Review

The literature relevant to ProcureNext falls into five threads: the economics of e-procurement, text retrieval, document understanding and evaluation with language models, explainable recommendation, and secure, auditable transaction systems.

2.1.1 E-procurement and its outcomes

Studies of public e-procurement consistently find that moving tendering online widens participation and reduces collusion. Lewis-Faupel et al. studied e-procurement roll-outs in India and Indonesia and found that it improved the quality of public works and increased the share of contracts won by firms from outside the region [7]. Neupane et al. identify transparency, monitoring and reduced personal contact between bidders and officials as the mechanisms through which e-procurement lowers corruption [8]. Vaidya et al. list the critical success factors as end-user uptake, supplier adoption, business-process re-engineering and security [9]. UNCITRAL’s Model Law codifies the same principles of open notice, equal treatment and a recorded process [10]. What this literature establishes is that *digital submission and a verifiable record* are the key interventions. What it leaves open is private-sector adoption, where no regulator mandates the platform, so the platform must earn adoption through convenience and intelligence rather than law. This motivates the “intelligent assistance” objective (O4).

2.1.2 Lexical, dense and hybrid retrieval

Classical retrieval ranks documents by term statistics. BM25 and its relatives remain strong baselines [11], and PostgreSQL’s `ts_rank_cd` applies the same principle over stemmed lexemes. Their known weakness is *vocabulary mismatch*: a vendor searching for “cellular handsets” will not find a tender for “smartphones”. Dense retrieval instead maps queries and documents into a shared embedding space. Sentence-BERT [12] made this practical for sentence-level semantic similarity, and dense passage retrieval [13] showed it can beat BM25 on open-domain QA.

MiniLM distils large transformers into small, fast students [14]; the resulting `all-MiniLM-L6-v2` produces 384-dimensional vectors on a CPU in milliseconds and scores competitively on the MTEB benchmark [15]. Approximate nearest-neighbour search with HNSW graphs [16], as implemented in `pgvector` [17], makes vector search scale inside a relational database. Dense retrievers, however, can miss exact identifiers and rare terms that lexical search handles well. Reciprocal Rank Fusion (RRF) [18] combines ranked lists by position alone, which avoids calibrating incomparable score scales, and routinely outperforms either component. *Gap*: this literature evaluates on web and QA corpora. We found no evaluation on short, domain-specific procurement notices, which is what Section 5.1 provides.

2.1.3 Document understanding and LLM-assisted evaluation

Procurement is document-heavy. Layout-agnostic PDF text extraction followed by rule-based field extraction is precise for fixed templates but brittle. Large language models (LLMs) with structured-output constraints generalise across layouts. Retrieval-augmented generation grounds them in source text [19]. Using an LLM as a *judge* of free-text quality correlates well with human ratings but shows position and verbosity biases [20]. The literature therefore supports LLMs as *assistants* whose outputs are persisted, explained, and overridable, not as decision makers. This supports our design of a hybrid pipeline (a deterministic parser as a fallback to the LLM, and LLM rubric scores as one weighted component of the evaluation, with a *needs_review* state).

2.1.4 Explainable recommendation

Trust in automated recommendations depends on users understanding them [21]. In a procurement setting an unexplained “top vendor” list would also raise fairness concerns. Simple additive models whose per-factor contributions are shown to the user are a recognised interpretable alternative to black-box rankers, and this is the approach ProcureNext takes for vendor recommendations (Section 4.2).

2.1.5 Secure, auditable transaction systems

Role-based access control (RBAC) is the standard model for enterprise authorisation [22,23]. For accountability, Haber and Stornetta showed how chaining hashes timestamps documents so that back-dating is detectable [24]. Schneier and Kelsey applied hash chains to audit logs [25], and Crosby and Wallach used Merkle trees for efficient tamper-evident logging [26]. Authenticated encryption with AES-GCM [27] provides both confidentiality and integrity for stored messages. These are mature techniques, but they are rarely combined in tendering systems, where audit trails are usually plain database tables that an administrator can edit.

Synthesis

The literature establishes that (i) digital submission and a verifiable record drive the benefits of e-procurement; (ii) hybrid lexical-plus-dense retrieval with rank fusion is the most robust search design; (iii) LLMs are useful but must stay subordinate to human judgement; and (iv) hash chains and Merkle trees give practical tamper evidence. It leaves open how to integrate these into a single affordable system for private-sector tendering in an emerging market, and how the retrieval methods perform on procurement text. ProcureNext addresses both.

2.2 Existing Solutions and Technologies

Bangladesh e-GP (eprocare.gov.bd). The national system, developed under the World Bank-supported Public Procurement Reform Project and run by BPPA, covers e-tendering and e-contract management [2]. We made a structured study of its public, unregistered-user surface, and this became the basis of our requirements. It offers general information pages; live e-tenders with full notices (procuring entity, method such as OTM or RFQ, eligibility, key dates, tender security, lot details); Annual Procurement Plans filterable by ministry, district and budget type; awarded contracts; a debarred-tenderer list; registration and coverage statistics; and advanced search by ministry, procurement nature, method, ID and date ranges. *Strengths:* strong transparency, a complete audit trail, national scale, and legal standing. *Limitations for private use:* it is closed to private buyers; workflows are fixed by regulation, with no buyer-controlled visibility or negotiation; search is keyword and filter based with no semantic matching; there is no vendor recommendation; communication is formal and limited; and the user interface is dense and form-heavy.

Private tender aggregators. bdtender.com, tenderbazar.com and AllTender.com (our industry partner's product) collect public and private tender notices, largely from newspapers, and sell subscription access to vendors. tendersinfo.com does the same internationally [28–31]. *Strengths:* broad coverage and familiarity to vendors. *Limitations:* they are notice boards. Vendors still buy documents and submit physically, buyers get no evaluation or audit support, and discovery is by exact keyword.

ERP and global e-sourcing suites. SAP Ariba, Oracle Procurement Cloud and the purchase/tender modules of open-source ERPs such as Odoo support RFQs inside one company's supply chain. *Strengths:* deep integration with inventory and accounts. *Limitations:* licensing and implementation costs beyond most Bangladeshi SMEs; they are not marketplaces, so they cannot help a buyer reach unknown vendors; and they are not adapted to local documents (trade

licence, TIN, VAT/BIN, bid security conventions).

Table 2.1: Comparison of existing solutions with ProcureNext

Aspect	e-GP (public)	Private aggregators	ERP / e-sourcing	ProcureNext
Target	Government	Private vendors	Single enterprise	Private buyers and vendors
Online bid submission	Yes	No	Internal only	Yes, with documents
Buyer-controlled visibility	No	n/a	n/a	Public / enlisted-only
Search	Keyword + filters	Keyword	Keyword	Hybrid semantic (RRF)
PDF notice import	No	No	No	Yes (LLM + rule parser)
Vendor recommendation	None	None	None	Explainable, 4 factors
Bid evaluation support	Committee matrix	None	Manual	Side-by-side + AI scoring
Messaging	Formal, limited	E-mail/phone	Internal	Encrypted tender chats
Audit trail	Strong	None	Varies	Hash-chained, verifiable
Reputation	Debarment list	None	None	Mutual buyer–seller reviews
Revenue model	Fees	Subscription	Licence	Usage-based tokens

Gap addressed. As Table 2.1 shows, no existing product combines e-GP-grade transaction integrity (online submission and an audit trail) with private-sector flexibility (buyer-controlled visibility, organisations with internal roles, messaging) and modern intelligence (semantic search, PDF import, explainable evaluation). ProcureNext is designed to fill that gap. Its integration of e-GP-format PDF import also means that buyers can bring notices already written in the national format onto the platform in one step.

Candidate technologies. The technology options examined were Node.js/Express, Django and FastAPI for the back end; Next.js and Vue/Nuxt for the front end; PostgreSQL with pgvector, Elasticsearch/OpenSearch, and dedicated vector databases (Pinecone, Milvus, Weaviate) for search; sentence-transformer models and hosted embedding APIs for semantic representation; Celery/Redis, RabbitMQ and Kafka for background work; and SSLCommerz as the dominant Bangladeshi payment gateway. They are compared in Sections 3.3 and 4.3.

2.3 Market and User Research

Market signals. The e-GP figures show that Bangladeshi firms are ready to tender digitally. More than 143,000 registered tenderers use it, the count grew by 19% in the year to November 2025, and monthly volume peaked at 20,000 tenders [1]. Many of these same firms also sell to private buyers, but through the manual channel described in Section 1.1. The existence of several paid aggregators (Section 2.2) shows that vendors already pay for private-tender *information*. ProcureNext extends the value offered from information to transaction.

Target users. Primary buyers are mid-to-large private organisations with recurring formal purchasing: manufacturers (RMG, pharmaceuticals, FMCG), real-estate and construction firms, private hospitals and universities, and NGOs with donor-mandated competitive procurement. Primary vendors are suppliers and contractors, including SMEs outside Dhaka, who most benefit from remote submission.

User research conducted. Requirement engineering took place during February 2026. (1) We analysed e-GP feature by feature (Section 2.2) to identify what private users would expect by analogy. (2) We reviewed the private aggregators. (3) We prepared research questions and a focus-group protocol, and reviewed our draft requirements with the industry mentor from Advanced Information Systems. This review produced several concrete requirements: the organisation-centric account model with a master account holder (FR-04, FR-06); buyer-defined required-document lists; enlisted-vendor (restricted) tenders (FR-12); the rule that inter-company chats include each company's owner (FR-07); and physical contract signing with in-system confirmation (FR-13). The requirement set was iterated and frozen on 24 February 2026 (Section 3.1).

2.4 Feasibility Study

Technical. Every core capability maps onto mature open-source technology: FastAPI and asyncpg for an asynchronous API; PostgreSQL for ACID transactions, full-text search (GIN indexes) and vectors (pgvector); all-MiniLM-L6-v2 for CPU-only embeddings; Celery/Redis for background work; and Next.js for the UI. The riskiest element, CPU-only semantic processing, was de-risked early. Our planning estimate was about 0.2 s to embed a page on a 4-core CPU. Measured embedding cost in the final system is 7 ms for a title and 28 ms for a two-thousand-character description (Section 5.1).

Economic. The inception-phase infrastructure plan, priced from the Bangladesh Data Center Company Limited (BDCCL) catalogue, sized a production node for 1,000 users at 4 OCPU, 16 GB RAM, 40 GB block and 50 GB object storage, at **BDT 18,288 per month**. Avoiding a GPU instance (roughly BDT 255,000 per month) was decisive; it is possible because the embedding model runs on CPU. Storage sizing: at an average of 2 MB per PDF, 50 GB holds about 25,600 documents, enough for 1,000 users uploading 25 documents each. Section 6.1.3 shows that the token model can cover this cost.

Operational. Buyers and vendors already use e-GP and web portals, so the interaction model is familiar. Organisation owners manage their own members, which keeps administrator workload limited to KYC verification and pricing.

Legal. Private procurement is governed by contract law rather than the PPA, which gives the freedom to offer flexible workflows. Electronic records are admissible under the ICT Act 2006 [32]. Keeping the binding contract signature offline avoids depending on the still-evolving digital-signature infrastructure.

Schedule. With five members over roughly eight months, and with modules that could be developed in parallel behind a shared schema, the scope of Section 1.4 was judged achievable. Section 6.1.1 reports the actual schedule.

Chapter 3

Requirements Analysis

3.1 Project Objectives and Constraints

3.1.1 Stakeholders

Table 3.1 lists the stakeholders, what each expects, and where their expectations conflict. Resolving these conflicts drove several design decisions.

3.1.2 Functional requirements

The functional requirements were consolidated on 24 February 2026 after the e-GP study and the mentor review. Table 3.2 summarises them with their MoSCoW priority [33]; the full acceptance criteria are in Chapter A.

Table 3.2: Functional requirements (baseline of 24 February 2026)

ID	Requirement	Summary	Pri.
<i>Unregistered user (public)</i>			
FR-01	General information	About, policies, legal, news, events, help pages without login	M
FR-02	Browse public tenders	Summaries of active public tenders; details and files need login	M
FR-03	Advanced semantic search	Semantic matching plus filters over active tenders	M
FR-04	Registration	Master account holder with NID and organisation documents (trade licence, TIN, VAT); members join under the master account; buyer, seller or both	M
<i>Registered users (common)</i>			
FR-05	Authentication	Login, logout, e-mail password reset, optional 2FA/OTP	M

ID	Requirement	Summary	Pri.
FR-06	Profile and affiliation	Owner invites members and assigns roles; join by organisation code	M
FR-07	Messaging and notifications	Intra-company DMs and groups; tender-wise inter-company groups including both owners; event notifications	M
<i>Buyer</i>			
FR-08	Tender creation and management	Create, edit, cancel, schedule, publish; budget, deadline, category, visibility, required documents, bid bond; single-item or packaged lots	M
FR-09	Vendor recommendations	Ranked vendors with explanations	M
FR-10	Bid comparison and evaluation	Side-by-side comparison of pricing and documents	M
FR-11	Notice of award	Select a seller, issue NOA, seller accepts; losers refunded	M
FR-12	Restricted invitations	Tenders visible only to enlisted vendors	S
FR-13	Contract draft and completion	Contract recorded; physical signing confirmed in system	M
FR-14	Seller performance reviews	Buyer reviews seller after completion	S
<i>Vendor</i>			
FR-15	Tender feed	Personalised dashboard of relevant tenders	M
FR-16	Bid submission	Technical/financial proposal, buyer-required documents, bid bond	M
FR-17	Bid status and history	Submitted, under evaluation, awarded, rejected; timestamped	M
<i>Financial</i>			
FR-18	Credit points	Purchase and manage tokens; balance and history	M
FR-19	Bid-bond transactions	Hold and refund bid bonds	M
FR-20	Publishing cost configuration	Global pricing with admin adjustments	M
<i>Administration</i>			
FR-21	Admin dashboard	Verify documents, manage users, configure pricing	M
FR-22	Audit trail	All critical actions logged, tamper-evident	M
FR-23	Disputes	Dispute lifecycle	C

ID	Requirement	Summary	Pri.
FR-24	Reporting and analytics	Exportable reports	C
	<i>External integrations</i>		
FR-25	Payment gateway	At least one secure gateway	M
FR-26	Identity verification APIs	Third-party NID/TIN verification	C

M = Must, S = Should, C = Could (marked “not sure” in the baseline).

3.1.3 Non-functional requirements

3.1.4 Constraints

- **Time:** one academic session, from requirements in February 2026 to submission on 29 September 2026, with course work in parallel.
- **Budget:** a student project with no funding. Production must fit a single commodity node (about BDT 18,000 per month) with no GPU; development must rely on free tiers and open-source software.
- **Team:** five developers with differing prior experience; the architecture must let them work in parallel with low coordination cost.
- **Data:** no public dataset of private tenders exists. Real e-GP notices are used for parsing, and a curated benchmark is used for retrieval (Section 5.1).
- **Platform:** web-first (desktop and mobile browsers); Linux servers; English-language documents.
- **Regulatory:** KYC documents required by Bangladeshi commercial practice; binding contracts signed physically.

3.2 Regulatory Requirements and Standards

Table 3.4 lists the regulations, standards and codes of practice that apply, why each applies, and how the design complies.

Table 3.1: Stakeholders, expectations and conflicts

Stakeholder	Expectations	Conflicts and resolution
Buyer organisation (owner, procurement officers, finance, viewers)	Reach many qualified vendors; control who sees a tender; compare bids quickly; defensible award record	Wants to see everything about vendors, while vendors want their pricing hidden from rivals. <i>Resolved:</i> bids are visible only to the buyer; competitors never see each other's bids.
Vendor organisation (incl. SMEs)	Discover relevant tenders; submit remotely and safely; fair, explainable evaluation; low cost to bid	Wants free access, but platform sustainability needs revenue. <i>Resolved:</i> free browsing, with a small token cost only for bidding.
Organisation members	Clear role, limited but sufficient privileges	Owner wants control; members want autonomy. <i>Resolved:</i> four fine-grained roles.
Unregistered public	Read-only overview of active public tenders and platform information	Transparency vs. buyer confidentiality. <i>Resolved:</i> public users see summaries only; documents and financials require login (FR-02).
Platform administrator	Verify KYC documents, set pricing, keep the platform trustworthy	Needs visibility for moderation, but must not alter history. <i>Resolved:</i> admin actions are themselves audited in a hash chain.
Industry partner (AIS / AllTender.com)	A product aligned with real private tendering practice and commercially viable	Feature breadth vs. time-to-market. <i>Resolved:</i> MoSCoW prioritisation (Table 3.2).
Payment provider (SSLCommerz)	Server-side validated transactions	None; handled via a gateway abstraction.
Regulators (NBR, BTRC, courts)	Legitimate, KYC-verified businesses; admissible records; data protection	Strict KYC vs. SME onboarding friction. <i>Resolved:</i> documents are uploaded once per organisation and reused for every bid.

Table 3.3: Non-functional requirements and measurable targets

ID	Quality	Requirement / target
NFR-1	Security	Authenticated access for all non-public operations; passwords hashed with bcrypt; tokens in HttpOnly cookies; RBAC on every protected endpoint; protection against unauthorised modification.
NFR-2	Confidentiality	Private messages encrypted at rest; bids visible only to the tender's buyer; restricted tenders hidden from non-enlisted vendors.
NFR-3	Auditability	Every state-changing request logged with actor, time, IP and before/after values; tampering detectable.
NFR-4	Performance	Interactive pages and search respond well under 1 s; long-running AI work never blocks a request thread (asynchronous jobs).
NFR-5	Availability	Public portal available 24/7; component failures degrade gracefully (e.g. search falls back to full-text if the ML service is down).
NFR-6	Scalability	Support growth in tenders and users; stateless API and ML containers scale horizontally.
NFR-7	Usability	Simple navigation for non-technical users; responsive layout; consistent, accessible UI components.
NFR-8	Maintainability	Modular code by domain; automated tests; CI on every change; reproducible containers.
NFR-9	Privacy	Tender embeddings computed on our own infrastructure; minimal personal data; documents served via authorised, time-limited links.

Table 3.4: Regulatory requirements and standards

Regulation / standard	Why it applies	How the design complies
Public Procurement Act 2006 and Rules 2008 [5, 6]	Not binding on private buyers, but the accepted national model of fair tendering	Adopted its concepts and vocabulary: procurement nature and method (OTM, RFQ, RFP, . . .), tender security, eligibility of tenderer, opening after closing, awarded-contract record, debarment-style reputation.
UNCITRAL Model Law on Public Procurement [10]	International principles of transparency and equal treatment	Equal access to public tenders, recorded process, no bid visible to competitors.
ICT Act 2006 (amended 2013) [32]	Legal recognition of electronic records; offences of unauthorised access	Electronic audit records with integrity proofs; access control and logging.
Contract Act 1872	Formation of binding contracts	Platform records award and contract confirmation; signature happens physically (FR-13).

Regulation / standard	Why it applies	How the design complies
VAT and SD Act 2012; Income Tax law; trade-licence rules	Legitimacy of businesses	KYC documents (trade licence, TIN, VAT/BIN) required from the master account and verified by an administrator before trading.
Cyber Security Act 2023 and data-protection principles [34]	Protection of personal data (NID, phone, e-mail)	Data minimisation, authorised access to documents, encrypted messages, no sale or sharing of data; GDPR principles used as the benchmark in the absence of a comprehensive enacted Bangladeshi data-protection statute.
OWASP Top 10 (2021) and ASVS [35, 36]	Web-application security baseline	Parameterised SQL (asynpg) against injection (A03); bcrypt and JWT expiry against broken authentication (A07); RBAC dependencies against broken access control (A01); secrets in environment files, never committed (A05); audit logging (A09).
NIST SP 800-63B [37]	Password and authenticator management	bcrypt hashing with per-password salt; single-use, 30-minute password-reset tokens from a CSPRNG; generic response to avoid account enumeration.
NIST SP 800-38D / FIPS 180-4 [27, 38]	Authenticated encryption; secure hashing	AES-256-GCM for stored messages; SHA-256 for audit hash chains and Merkle roots.
RFC 7519 (JWT) [39]	Token-based session format	Signed access tokens (60 min) and refresh tokens (7 days) in HttpOnly cookies.
PCI DSS scope minimisation [40]	Card payments	No card data touches our servers; payments go through a hosted gateway with server-side validation.
WCAG 2.1 AA [41]	Accessibility of the web UI	Semantic HTML, keyboard-operable components, sufficient contrast in the enterprise theme, text labels for icons (Section 6.5.1).

Regulation / standard	Why it applies	How the design complies
ISO/IEC 25010 [42]	Software product quality model	NFRs in Table 3.3 organised along its characteristics (security, performance efficiency, reliability, usability, maintainability).
ISO/IEC 27001:2022 [43]	Information-security controls	Informed our controls on access control, logging, cryptography and secure development; formal certification is out of scope.
ACM / IEEE / IEB codes of ethics [44, 45]	Professional conduct	See Section 6.5.4.

3.3 Formulation of Solutions

We formulated three genuinely different solutions that could each meet most of the requirements.

Solution A: Customise an existing ERP/e-sourcing product. Extend an open-source ERP (e.g. Odoo’s purchase and tender modules) with a public portal, multi-company access and keyword search. This is quick to demonstrate and inherits accounting features. But the ERP’s data model is single-enterprise, so turning it into a two-sided marketplace with cross-organisation visibility rules fights the framework; semantic search and ML must be bolted on; and customisation is locked to the vendor’s upgrade cycle.

Solution B: Cloud-native microservices with managed AI services. Separate services for auth, tenders, bids, payments, messaging and search; Kafka for events; a managed vector database (Pinecone/Milvus); hosted embedding and LLM APIs for all intelligence; Kubernetes for orchestration. This is the most independently scalable option. It is, however, operationally heavy for a five-person team; it requires distributed transactions (sagas) for flows such as “accept bid $\Rightarrow$ reject others $\Rightarrow$ create award $\Rightarrow$ debit tokens”; it sends all tender text to third parties; and it has a recurring cost far above the budget.

Solution C: Modular monolith with a separate stateless ML service (selected). A FastAPI application organised into domain modules sharing one PostgreSQL database; pgvector and PostgreSQL full-text search in the same database; a separate, stateless Python ML service for PDF parsing, embeddings and bid scoring; Celery with Redis for asynchronous jobs; a Next.js

front end; all in Docker. Cross-module workflows are single ACID transactions. The ML service scales independently and can be moved to a GPU host later without touching business logic. Embeddings are computed locally; an external LLM is used only where it adds clear value, with a deterministic fallback.

Table 3.5: Weighted evaluation of candidate solutions (scores 1–5)

Criterion (source)	Weight	A: ERP	B: Microservices	C: Modular monolith
Fit to two-sided marketplace (FR-02, 12, 15)	0.20	2	5	5
Transactional integrity (G2, G3)	0.15	4	2	5
Semantic search and ML integration (FR-03, 09, 10)	0.15	2	5	4
Data privacy (NFR-9)	0.10	4	2	4
Operating cost within budget	0.15	3	1	5
Delivery risk for a 5-person team	0.15	3	1	4
Future scalability (NFR-6)	0.10	2	5	4
Weighted total	1.00	2.80	3.05	4.50

Conclusion. Solution C scores highest (Table 3.5). It gives up only a small amount of theoretical scalability relative to B, and even that is recovered by the independent ML service and stateless containers. Solution B’s advantages matter only at a scale that the budget cannot yet pay for, and Solution A cannot naturally express the marketplace. Within C, the same method selected (i) PostgreSQL + pgvector over a dedicated vector database, which removes a second datastore at no measurable latency cost at our scale (Section 5.1); and (ii) hybrid RRF search over pure lexical or pure dense search (Section 2.1).

Chapter 4

System Design and Implementation

4.1 Functional Design and Sub-problem Partitioning

Figure 4.1 shows the deployed architecture. It follows Solution C of Section 3.3: a Next.js presentation tier, a FastAPI modular monolith, a separate stateless ML service, a Celery worker, and a single PostgreSQL database that holds relational data, full-text indexes and vector embeddings.

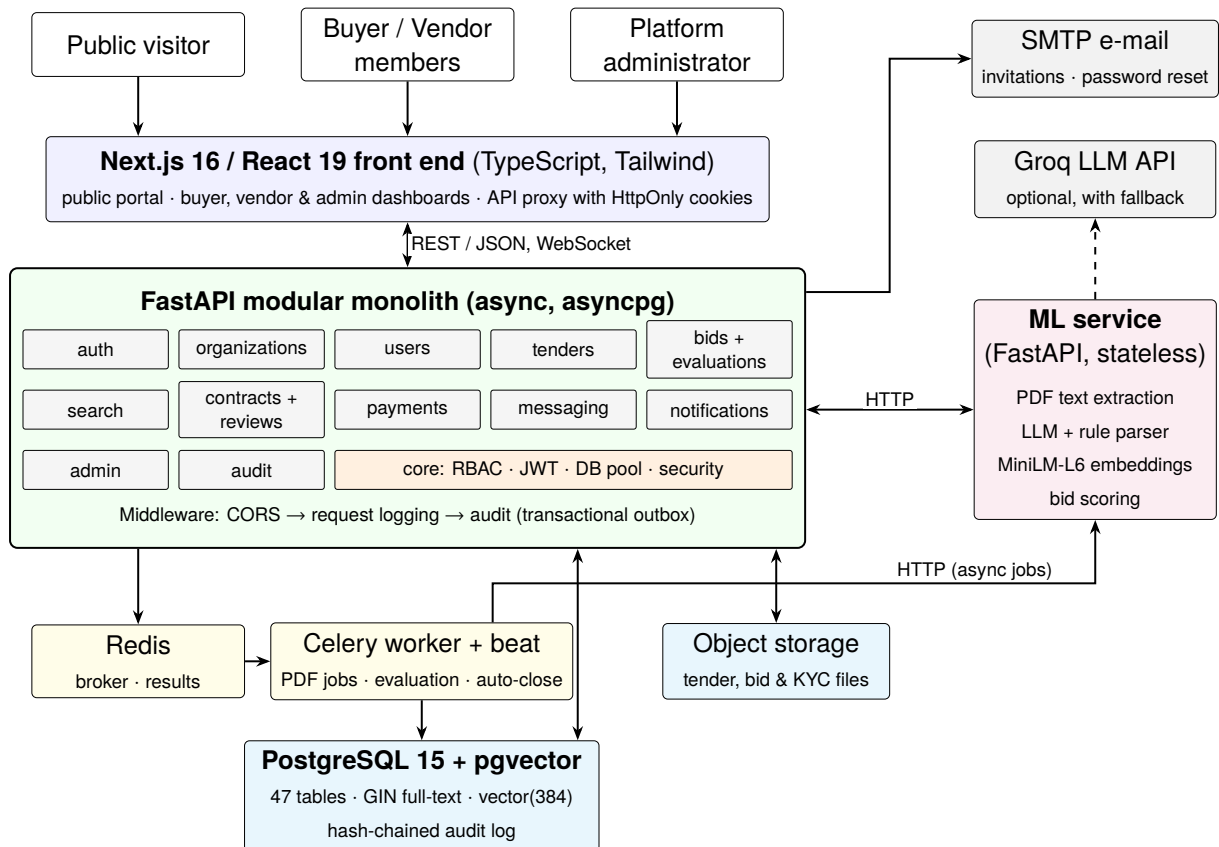


Figure 4.1: ProcureNext system architecture as deployed. Solid arrows are synchronous calls; the dashed LLM call is optional, with a deterministic fallback.

4.1.1 Sub-problems and modules

The problem splits into the sub-problems below. Each maps to one backend module (a `router.py/service.py/schemas.py` package) and to front-end pages. Table 4.1 gives each module's responsibility, requirements and main interactions.

Table 4.1: Module partitioning

Module	Responsibility	FRs	Interacts with
auth	Login, registration via invitation, JWT issue/refresh, password reset by e-mail token	04, 05	users, SMTP, audit
organi- zations	Master-account sign-up with KYC files, unique join code, member invitations by e-mail, role changes, enlisted-vendor lists, public profiles, org search	04, 06, 12	admin, storage, notifications
users	Own profile, password change, personal NID documents	05, 06	auth, storage
admin	Verify pending master accounts and documents; token pricing and packages	20, 21	organizations, payments
tenders	Draft/publish/edit/cancel/delete; lots and required documents; PDF import job; public and seller listings with visibility filter; vendor recommendations; auto-close	02, 08, 09, 12, 15	ML service, Celery, payments, notifications
search	Hybrid full-text + semantic search with RRF	03, 15	ML service, tenders
bids + evaluation	Submit/update/withdraw bids with documents; buyer comparison; accept bid and award; AI evaluation runs	10, 11, 16, 17	payments, ML service, Celery, notifications
contracts	Contract completion; mutual buyer↔seller reviews and reputation	13, 14	bids, organizations
payments	Token balance, packages, purchase, append-only ledger	18, 20, 25	tenders, bids, admin

Module	Responsibility	FRs	Interacts with
messaging	Encrypted DMs, intra-company groups, tender-scoped inter-company rooms; members/admin management; WebSocket delivery	07	notifications
notifi- cations	In-app notifications, unread counts, real-time push	07	all workflow modules
audit	Transactional outbox, hash chain, integrity verification, Merkle archives, entity trails	22	middleware, admin
core	RBAC dependencies, security utilities, DB pool, pagination, exceptions	all	all
ML service	PDF text extraction, LLM/rule field extraction, embeddings, bid scoring	03, 08, 09, 10	tenders, search, bids
Front end	27 pages: public portal, sign-up flows, buyer/vendor/admin dashboards, tender and bid workspaces, messaging	all	backend via proxy

4.1.2 How standards and safety shaped the design

- **OWASP A01 (access control):** authorisation is centralised in `core/permissions.py`, not scattered through handlers. FastAPI dependencies such as `require_role("Owner", "ProcurementOfficer")` guard each endpoint, and a single SQL fragment, `build_visibility_filter()`, is reused by listing, detail and search queries so that restricted tenders cannot leak through a forgotten code path.
- **OWASP A03 (injection):** all SQL uses `asyncpg` positional parameters; dynamic query parts are assembled from whitelisted fragments only.
- **NIST 800-63B and RFC 7519:** `bcrypt` password hashes; short-lived access tokens held in `HttpOnly` cookies by the Next.js proxy, so browser JavaScript never sees them; single-use reset tokens.
- **NIST 800-38D and FIPS 180-4:** message bodies are encrypted with AES-256-GCM before storage, and audit records are SHA-256-chained.
- **PCI DSS:** the purchase flow never collects card data.

- **Data protection:** files are served only through authorised endpoints; unregistered users see summaries only (FR-02).
- **Health and safety:** the platform's main safety contribution is removing the need to physically deliver bids, which eliminates the risk of intimidation at tender boxes (Section 6.3).

4.2 Design Refinement and Simulation

4.2.1 Use cases and data flow

Figure 4.2 shows the formal use-case model produced in the analysis phase. Figures 4.3 and 4.4 show the context-level and level-1 data-flow diagrams. The level-1 DFD decomposes the system into four processes: user/organisation verification, tender management with ML search, bidding and evaluation, and financials. These became module boundaries.

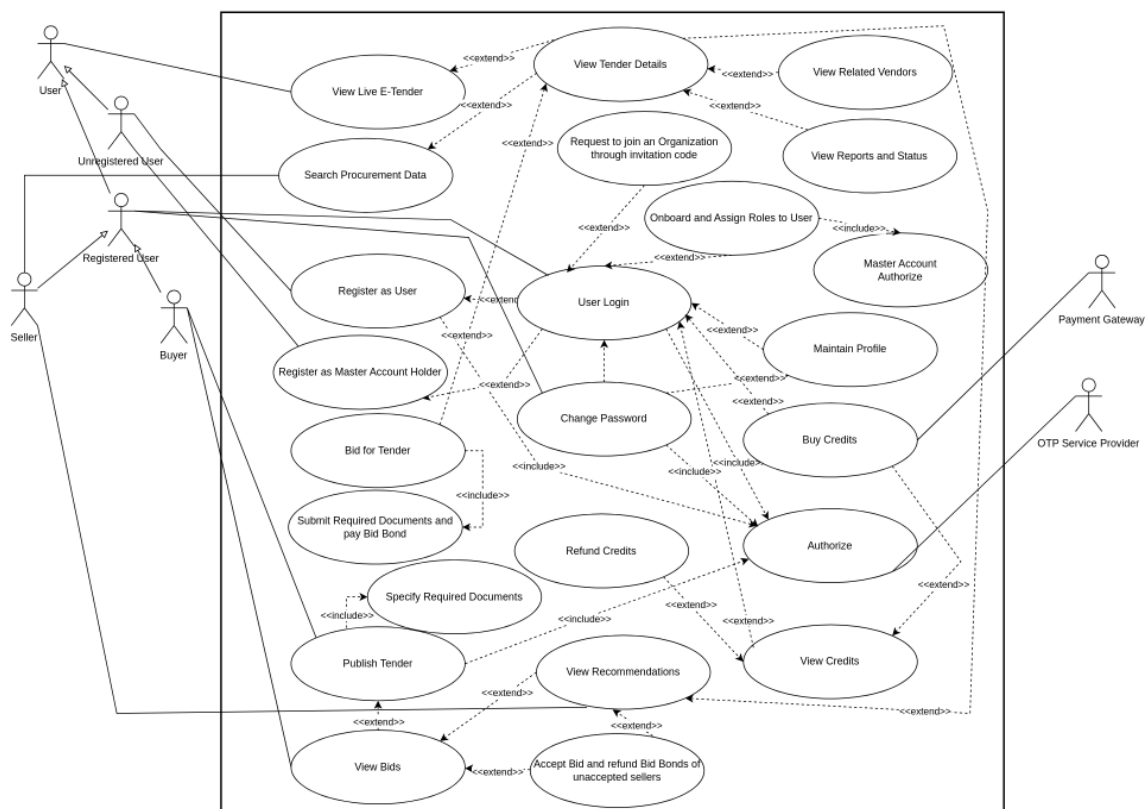


Figure 4.2: Use-case diagram (actors: unregistered user, registered user, buyer, seller, payment gateway, OTP provider)

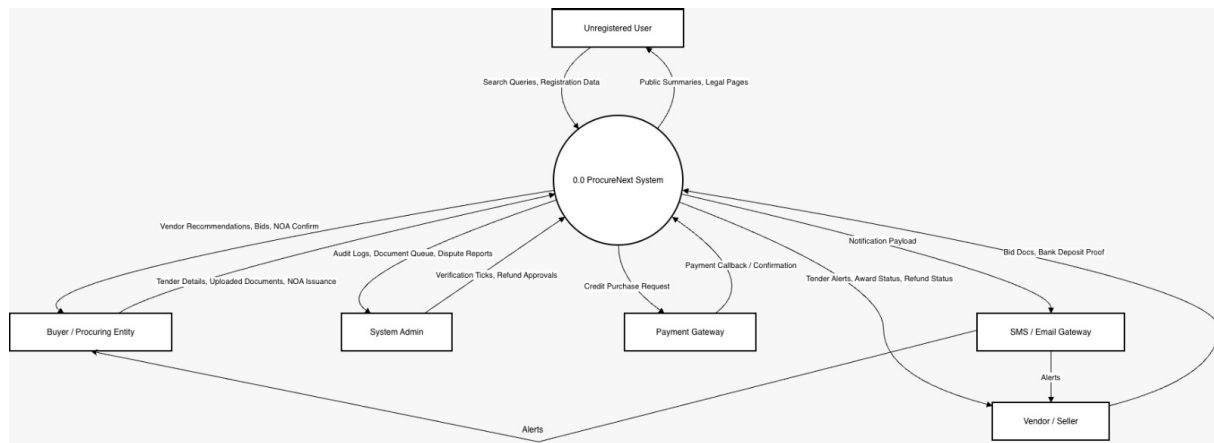


Figure 4.3: Level-0 (context) data-flow diagram

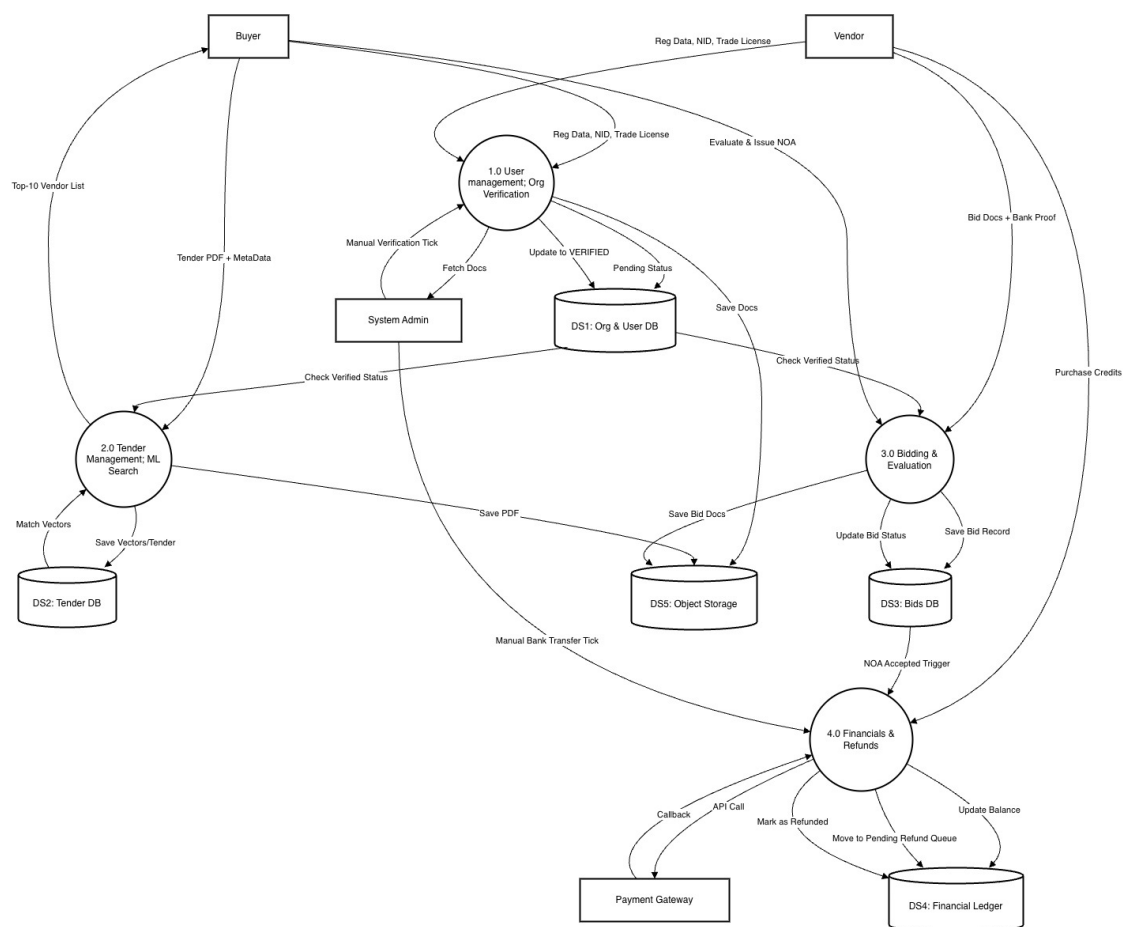


Figure 4.4: Level-1 data-flow diagram

4.2.2 Data model

The database has 47 tables in 11 groups: identity and verification; organisations, members, documents and enlistment; lookups (procurement nature, method, categories); tenders, lots, required documents, invitations and suggestions; bids, bid documents and securities; awards, contracts and performance/mutual reviews; payments, credit transactions, pricing and packages; notifications; messaging (private, group, tender rooms, participants, seen-receipts); evaluation runs and results; and audit (log, outbox, archives). Figure 4.5 shows the core entity–relationship design. Key decisions:

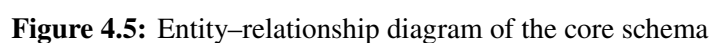
- **Organisation-centric ownership.** Tenders, bids and balances belong to organisations; actions are attributed to an `organization_employees` row, so both “which company” and “which person” are recorded.
- **One bid per vendor per tender**, enforced by `UNIQUE(tender_id, vendor_org_id)`.
- **Generated search column.** `search_vector` is a `GENERATED ALWAYS` tsvector (title weight A, description weight B) with a GIN index, so the full-text index can never go stale.
- **Append-only evaluation results.** Each “Evaluate bids” press creates a new run; results are never overwritten, preserving the model name, prompt version and weights used for each run.

4.2.3 Tender lifecycle

Figure 4.6 shows the tender state machine that guarantees G2. Publishing debits the buyer’s tokens in the same transaction as the state change. A Celery beat job runs hourly and moves every *Published* tender past its deadline to *Closed*. Accepting a bid atomically marks it *Accepted*, rejects all other pending bids, creates the award, and sets the tender to *Awarded*.

4.2.4 Interaction design

Interaction diagrams were drawn for nine key flows (master-account registration, role assignment, buy and refund credits, publish tender, bid, accept bid, send and accept NOA). Figure 4.7 shows the bid-submission flow: documents go to object storage, parsing is queued, and both parties are notified asynchronously.



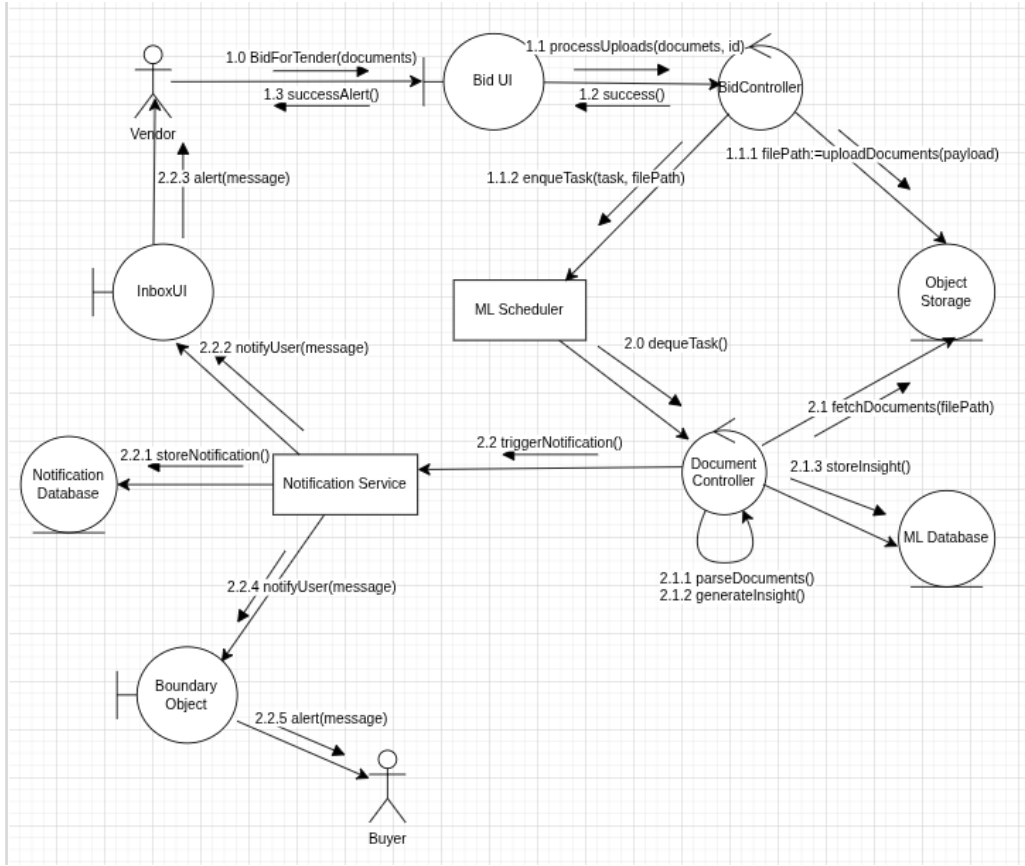


Figure 4.7: Collaboration diagram for bid submission

4.2.5 Algorithms

Hybrid semantic search (FR-03). For a query q , the lexical arm ranks tenders visible to the viewer that match $\text{plainto_tsquery}(q)$ by ts_rank_cd . The semantic arm embeds q with MiniLM and ranks tenders by cosine distance $\delta(q, t) = 1 - \frac{\mathbf{e}_q \cdot \mathbf{e}_t}{\|\mathbf{e}_q\| \|\mathbf{e}_t\|}$, keeping only $\delta < 0.8$ so that a nonsense query returns few results instead of a page of nearest neighbours. Each arm contributes up to 50 candidates, which are fused by RRF [18]:

$$\text{score}(t) = \sum_{a \in \{\text{lex}, \text{sem}\}} \frac{1}{k + \text{rank}_a(t)}, \quad k = 60, \quad (4.1)$$

where a tender absent from an arm contributes 0 from that arm. RRF needs no normalisation between the unbounded ts_rank_cd and the bounded cosine. If the ML service is unreachable, search degrades to the lexical arm alone (NFR-5).

Explainable vendor recommendation (FR-09). For each candidate vendor v of tender t (excluding the buyer, and excluding non-enlisted vendors for restricted tenders):

$$S(v, t) = 100 \left(0.35 c + 0.30 \frac{\bar{r}}{5} + 0.20 e + 0.15 z \right), \quad (4.2)$$

where $c = 1$ if v has bid in t 's category before (else 0.15), $\bar{r}$ is its average mutual-review rating (default 3), $e = 1$ if the buyer has enlisted v , and $z = 1$ if certifications are on file (else 0.35). Every non-trivial factor also produces a human-readable reason (e.g. "High mutual satisfaction rating (4.6/5.0 stars)"), and the top 15 are returned. Because the model is additive, the explanation is exact rather than a post-hoc approximation.

Smart bid evaluation (FR-10). When a buyer presses *Evaluate bids*, a run is queued to Celery; the ML service scores each bid x on four components in $[0, 100]$:

- *Financial* $F(x)$: with budget midpoint $m = (\beta_{\min} + \beta_{\max})/2$ and amount a ,

$$F = \begin{cases} \min(100, 80 + 0.4 \cdot 100 \frac{m-a}{m}) & a \leq m \\ \max(50, 80 - 100 \frac{a-m}{m}) & m < a \leq \beta_{\max} \\ \max(0, 50 - 100 \frac{a-\beta_{\max}}{\beta_{\max}}) & a > \beta_{\max} \end{cases}$$

Low bids are never penalised in the score. Instead, a bid below half the median of the tender's bids is *flagged* as a possible unrealistically low offer for the buyer to check.

- *Document compliance* $D(x)$: the percentage of the buyer's required documents that the bid supplied, with the missing ones listed.
- *Semantic relevance* $E(x)$: cosine similarity between the tender and bid embeddings, mapped from $[-1, 1]$ to $[0, 100]$.
- *LLM rubric* $L(x)$: the mean of clarity, completeness and feasibility scores, each with a one-line justification and a list of risk flags, obtained through structured output with up to three retries.

The composite score is

$$C(x) = 0.20 F + 0.20 D + 0.05 E + 0.55 L, \quad (4.3)$$

with weights stored per run so that any score can be reproduced. If the LLM fails, the bid is marked *needs_review* with no composite score. The system never invents a number, and the award itself is always a human decision.

Tamper-evident audit chain (FR-22). The audit middleware writes every POST, PUT, PATCH or DELETE request into an `audit_outbox` table without blocking the request. A processor then appends events to the log strictly in sequence, under an exclusive tail lock. For record i with

canonical (sorted-key) JSON payload P_i , timestamp τ_i and sequence number i :

$$p_i = \text{SHA256}(P_i), \quad h_i = \text{SHA256}(h_{i-1} \parallel p_i \parallel \tau_i \parallel i), \quad h_0 = 0^{64}. \quad (4.4)$$

Changing any field of any record changes p_i and hence every later h_j . Deleting or reordering records breaks the sequence. The verification endpoint recomputes the chain and reports each anomaly. Batches of signatures are periodically sealed into archives identified by their Merkle root [26], which gives a compact commitment that can be stored externally.

4.2.6 Design calculations

- **Vector storage:** one 384-dimensional float32 embedding is $384 \times 4 = 1,536$ bytes. 10^5 tenders need about 147 MiB of vectors, which fits comfortably in the 16 GB node's RAM together with an HNSW index. The first design used 768 dimensions, which would double this; the refinement below reduced it.
- **Embedding throughput:** measured median latency is 28 ms for a 2,000-character description (Section 5.1), so one core can embed about 35 tenders per second, far above the expected publishing rate.
- **Document storage:** 2 MB per PDF $\Rightarrow$ 25,600 documents in 50 GB (Section 2.4).
- **Request budget:** the search path is one embedding call (about 7 ms for a short query) plus two indexed SQL queries; the measured end-to-end median is 8 ms, far inside the 1 s NFR-4 target.

4.2.7 Design refinements

The design was verified and refined through prototypes and measurement:

1. **Back end: Node.js/Express $\rightarrow$ FastAPI.** The first skeleton (April 2026) was written in Express. Once the ML pipeline was prototyped in Python, keeping two languages meant duplicated models and an extra network hop for every AI call. The back end was re-based on FastAPI in June and the Node code removed in July. This unified validation (Pydantic) and made ML integration straightforward.
2. **Microservices $\rightarrow$ modular monolith + ML service.** The analysis-phase design proposed full microservices behind an API gateway. Prototyping the accept-bid flow showed the need for cross-domain atomicity, so domains were merged into one process with one database, and only the CPU-heavy, stateless ML work was kept as a separate service.
3. **Synchronous $\rightarrow$ asynchronous PDF import.** Parsing a notice with an LLM can take

several seconds. The import endpoint therefore returns `202 Accepted` with a Celery task ID, and the UI polls the job status, following the refined sequence design of the analysis phase.

4. **768 → 384 dimensions.** The schema initially reserved `VECTOR(768)`. Choosing all-MiniLM-L6-v2 for CPU efficiency led to a migration to `VECTOR(384)`, halving vector memory.
5. **Outlier detection: IQR → median fence.** With only a few bids per tender, the quartile fence was dragged down by one legitimately cheaper bid and masked a true low-ball. Comparing each bid to half of the median is robust at small sample sizes; this was verified on the worked example in Table 5.5.
6. **Similarity ceiling for semantic search.** Without a distance threshold, the vector arm always returns 50 “nearest” tenders, even for unrelated queries. The $\delta < 0.8$ ceiling was added and validated experimentally (Table 5.3).

4.3 Application of Tools

Table 4.2: Tools and technologies: justification and limitations

Tool	Why selected	Limitations
Python 3.12, FastAPI, Pydantic	Native language of the ML ecosystem; async I/O; automatic OpenAPI documentation; strict request/response validation	Single-threaded event loop: CPU work must be offloaded (done via Celery and the ML service)
asyncpg (raw SQL)	Fastest PostgreSQL driver for asyncio; full control over complex SQL (RRF, visibility filters)	No ORM migrations; schema changes are managed through versioned SQL scripts
PostgreSQL 15 + pgvector	ACID, JSONB, generated tsvector + GIN, and vector search in one engine; no extra datastore	Vector search quality and speed below specialised engines at very large scale
MiniLM-L6-v2 via sentence-transformers	384-d, CPU-fast, strong semantic quality; runs on our infrastructure (privacy)	English-centric; 256-token input window truncates long texts
pypdf	Pure-Python, dependency-free text extraction for generated PDFs	No OCR for scanned PDFs

Tool	Why selected	Limitations
Groq LLM via LangChain (structured output)	Low-latency inference for field extraction and rubric scoring; schema-constrained JSON	Third-party API; mitigated with a deterministic fallback and use only on tender and bid text
Celery + Redis	Mature task queue with retries and a periodic scheduler (beat)	Adds a broker to operate
Next.js 16, React 19, TypeScript, Tailwind 4	Server-side proxying keeps tokens in HttpOnly cookies; type safety; fast UI iteration	Rapid framework churn
AES-GCM (cryptography), bcrypt, python-jose	Vetted primitives for message encryption, password hashing and JWT	Encryption key held in the environment; a KMS would be stronger
Supabase-managed PostgreSQL	Managed backups and pgvector support without self-hosting	Vendor dependency; network latency from the app host
Docker, Docker Compose	Identical environments for development and production; one-command deployment	Single-host orchestration only
GitHub, GitHub Actions	Pull-request workflow; CI tests and CI-gated deployment	Free-tier runner minutes
pytest, httpx	Async API testing with dependency overrides and mocks	Mocks do not catch every schema drift
draw.io, PlantUML	ERD, DFD, use-case, collaboration and sequence diagrams	Diagrams maintained separately from code

4.4 Implementation

4.4.1 Scale of the implementation

The implementation consists of about 16,300 lines of backend Python (12 mounted modules, 99 API endpoints), about 690 lines in the ML service, about 17,600 lines of TypeScript/TSX across 27 front-end pages, and 9,300 lines of tests. Figures 4.8 to 4.11 show the implemented interface.

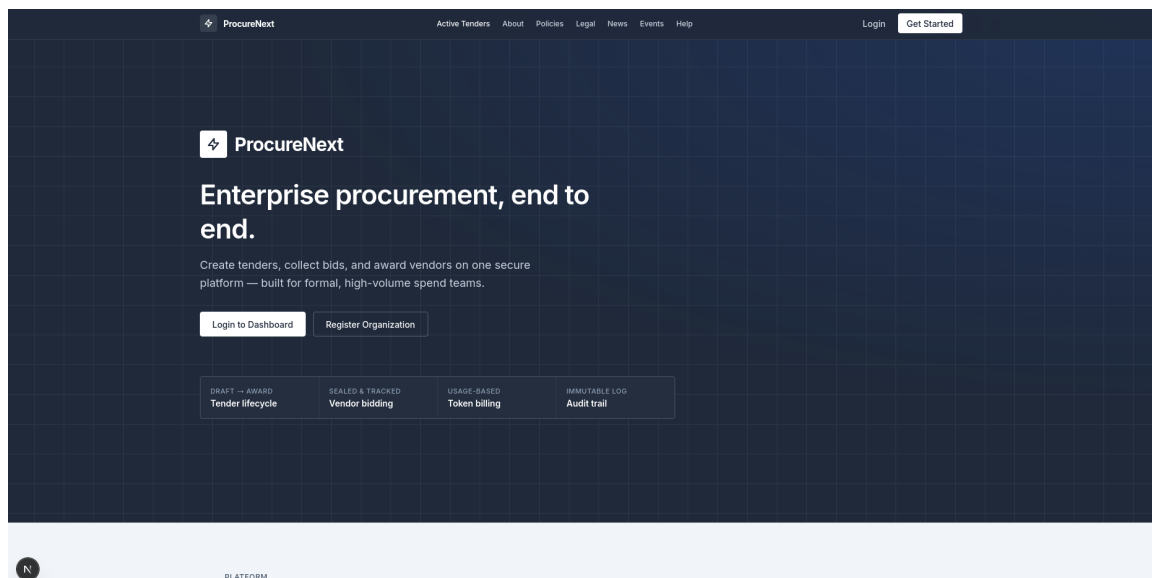


Figure 4.8: Public landing page

Figure 4.9: Buyer dashboard: tender portfolio with status filters, token balance and search

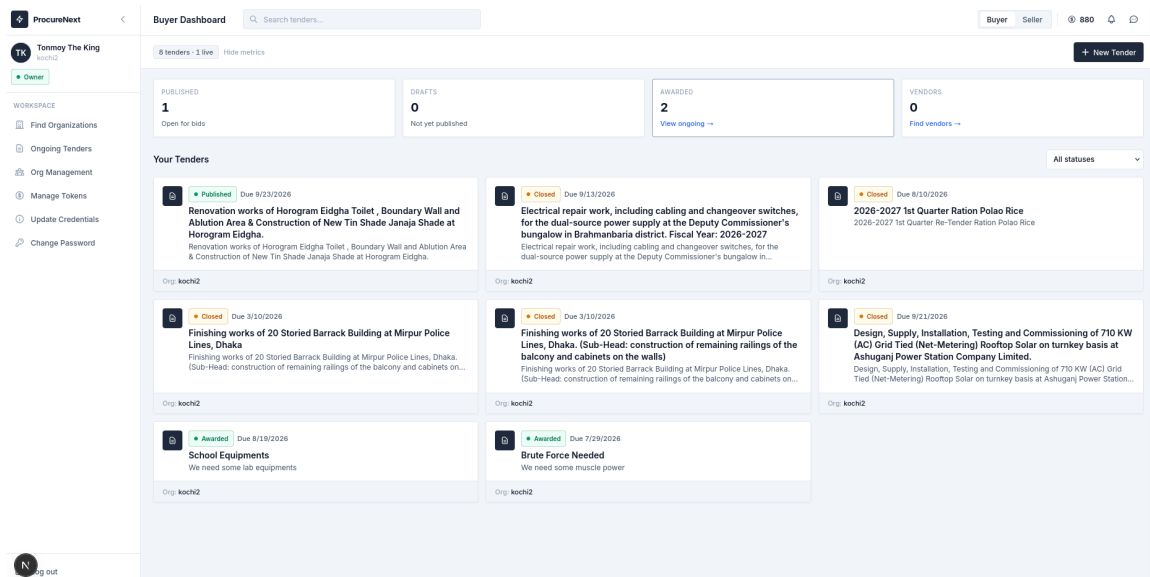


Figure 4.10: Tender creation with one-click import from an e-GP notice PDF, packaged lots, key dates and required specification documents

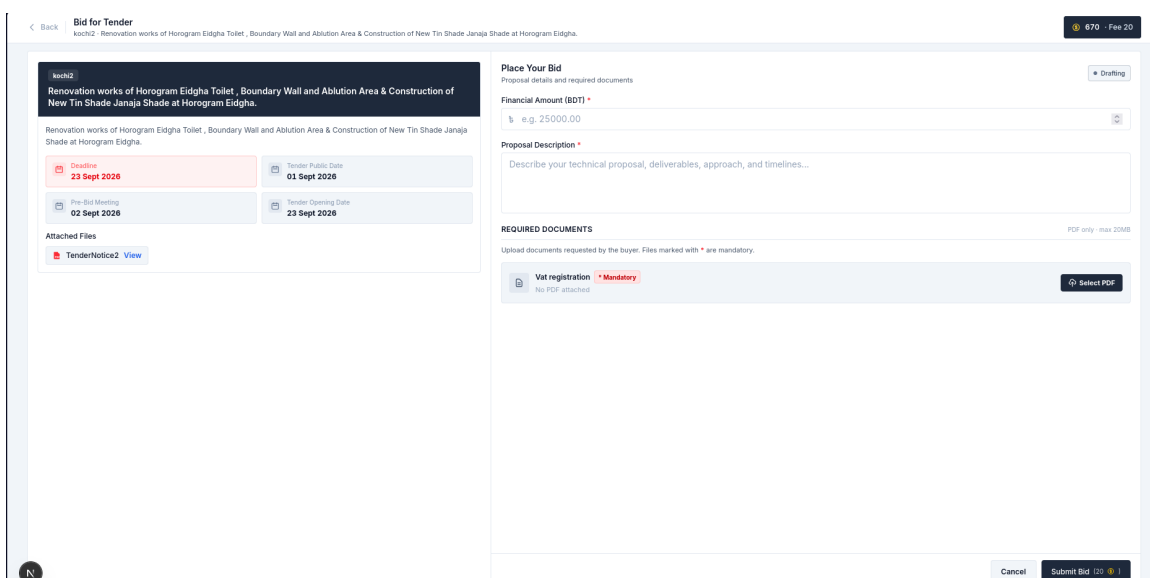


Figure 4.11: Vendor bid submission with financial amount, proposal and buyer-mandated documents

4.4.2 Key modules

Onboarding and RBAC. A master account holder registers the organisation, uploads NID and KYC documents, and waits for an administrator to verify them. The organisation receives a unique join code. Owners invite members by e-mail (Gmail SMTP) with expiring invitation tokens and assign *Owner*, *ProcurementOfficer*, *Finance* or *Viewer*. The RBAC engine exposes capability groups (tender managers, bid submitters, finance managers, organisation admins) as FastAPI dependencies.

Tenders. Buyers create single-item or packaged multi-lot tenders with procurement nature and method, budget range, eligibility, required documents, key dates and visibility. They can save drafts, publish (debiting tokens), edit (freely as a draft; while published only if there are no bids), withdraw, or delete. *Import from PDF* uploads an e-GP-style notice; a Celery task sends it to the ML service, which returns structured fields and an embedding that pre-fill the form.

Bidding, evaluation and award. Vendors submit a bid with amount, proposal and each required document, and can update or withdraw it before the deadline. Buyers see a comparison view with per-document compliance, trigger AI evaluation runs, and accept a bid, which rejects the others, creates the award and notifies every bidder. Contracts are completed and both parties leave mutual reviews, which feed the reputation used by recommendations.

Messaging and notifications. Direct messages, intra-company group chats, and tender-scoped inter-company rooms in which both organisations' owners manage membership, as required by FR-07. Bodies are encrypted with AES-256-GCM. Messages and notifications are delivered in real time over WebSockets, with unread counts.

Token economy. Each organisation starts with 250 tokens. The default pricing, which an administrator can change, is BDT 1 per token, 50 tokens to publish and 20 to bid, with packages of 100, 250, 500 and 1,000 tokens (the larger ones discounted up to 25%). Every movement writes a `credit_transactions` row with the balance after the change, inside the same transaction as the action that caused it.

4.4.3 Development process

- **Repository and branching.** One GitHub repository (hasanaranna/ProcureNext) with feature branches (e.g. `feature/RBAC`, `semanticSearch`, `SmartBidEvaluation`,

`forgetpassword`, `ui-overhaul`) merged through 37 pull requests; 134 commits from February to September 2026.

- **Code review.** Integration happened through pull requests, most of which were merged by a teammate after review; merges were spread across all five members (Section 6.2.2).
- **Testing.** 37 test modules and 346 test cases across every mounted module, including audit tamper detection and RBAC.
- **Continuous integration.** Every push and pull request to `main` runs the full `pytest` suite and a TypeScript type-check and Next.js production build (`ci.yml`).
- **Continuous deployment.** `deploy.yml` runs only when CI on `main` succeeds (`workflow_run`); it connects to the server over SSH, fetches `main`, and rebuilds the images sequentially to avoid running out of memory on the host.

4.5 Deployment

Environment. Production runs on a single AWS EC2 Linux instance with Docker Compose (`docker-compose.prod.yml`) and five containers: `frontend` (Next.js production build, exposed on port 80), `backend` (Uvicorn/FastAPI), `ml` (ML service with a persistent Hugging Face model cache), `celery_worker` (worker plus beat scheduler), and `redis`. Only the front end is exposed publicly; the back end, ML service and Redis are reachable only on the internal Docker network. PostgreSQL is hosted on Supabase (managed, with automated backups), reached through `DATABASE_URL`. Uploaded files persist in a named Docker volume.

Configuration and release. Secrets (database URL, JWT key, message-encryption key, SMTP and LLM credentials) live in `.env` files on the host and are never committed. A release is a merge to `main`: CI tests it, and on success CD rebuilds and recreates the containers (`up -d -force-recreate -remove-orphans`). Rollback is a revert commit through the same pipeline.

Monitoring and backup. The request-logging middleware writes a structured JSON line for each request with its method, path, status and latency. The audit subsystem exposes statistics (outbox backlog, chain length) and an on-demand integrity check. Database backups are handled by the managed provider.

Operation. During the course the team operates the deployment (DevOps owner: Tawhid Umar). The commercial path, with operation by the industry partner, is discussed in Section 6.1.4.

Resources consumed. One virtual machine running five containers. The ML container is the largest because of the PyTorch runtime and a model of about 90 MB; idle CPU use is near zero, since embedding runs only on publish, search and evaluation. There is no GPU. Section 6.4 assesses the environmental impact.

Chapter 5

Investigation and Evaluation

5.1 Experimental Investigation

The intelligent components carry the project’s main technical claims, so we tested them experimentally on the implemented code. Every experiment called the production modules directly (`ml/src/tender_parser.py`, `ml/src/bid_evaluator.py`), and the search experiment reproduced the exact SQL of `search/service.py` on a real PostgreSQL 16 + pgvector instance. All timings were measured on a 16-thread x86-64 laptop CPU with no GPU, which is a conservative stand-in for the 4-OCPU production target on a per-core basis.

Research questions.

- RQ1** Can structured tender fields be extracted reliably from real Bangladeshi e-GP notice PDFs without human input?
- RQ2** Does hybrid (lexical + semantic) search retrieve relevant tenders better than keyword search, especially when the vendor’s wording differs from the buyer’s?
- RQ3** Does the similarity ceiling suppress irrelevant results for off-topic queries?
- RQ4** Is CPU-only embedding fast enough for interactive use?
- RQ5** Does the financial scoring behave as designed, including outlier flagging at small sample sizes?

5.1.1 E1: Tender-notice parsing (RQ1)

Data. Three real tender notices downloaded from `eprocure.gov.bd`: (1) finishing works for a 20-storey barrack building, PWD Mirpur (Works, 3 pages); (2) repair of the water-supply system of the 250-bed General Hospital, Kushtia (Works, 3 pages); (3) supply of stone chips, sand and bitumen for road repair, DSCC (Goods, 2 pages). **Method.** We ran the deterministic rule-based parser, the path that always runs and serves as the fallback for the LLM path, on each PDF and

compared ten fields per notice against values read manually from the PDF. **Metric.** A field counts as correct only on an exact match of the value (dates to the minute).

Table 5.1: E1: field extraction from real e-GP notices (deterministic parser)

Field	Notice 1	Notice 2	Notice 3
Procurement nature (Works/Goods)	✓	✓	✓
Procurement method (Open Tendering Method)	✓	✓	✓
Title (package description, multi-line)	✓	✓	✓
Category (long, multi-line list)	✓	✓	✓
Eligibility of tenderer	✓	✓	✓
Publication date/time	✓	✓	✓
Closing date/time	✓	✓	✓
Pre-tender meeting date/time	✓	✓	✓
Opening date/time	✓	✓	✓
Tender security amount (in lot table)	–	–	–
Correct fields	9/10	9/10	9/10
Text extraction time (ms)	118	121	99
Field parsing time (ms)	14	0.3	0.3

Result. 27 of 30 fields (90%) were extracted exactly, in about 0.1 s per notice (Table 5.1). The multi-line fields, which are the tedious ones for a buyer to retype (titles, the category list of about 30 entries, and eligibility clauses), were all correct. All twelve dates were exact to the minute, even though e-GP places them in a two-column layout. The only miss was the tender-security amount, which e-GP puts inside the lot table rather than on a labelled line. In production this field comes from the LLM path and is shown for the buyer to confirm, so the system does not rely on the fallback for it. **Conclusion:** a buyer can bring an existing e-GP notice onto ProcureNext with nine of its ten key fields pre-filled, and none of them needs retyping.

5.1.2 E2: Hybrid semantic search vs. keyword search (RQ2)

Data. No public relevance-labelled corpus of Bangladeshi tenders exists, so we built a benchmark. It contains 48 tender notices in the style of private and e-GP tenders across 12 domains (IT, construction, energy, health, facilities, transport, agriculture, textiles, services), and 25 queries with relevance judgements made before running any system. Ten queries are *lexical*, meaning they share key words with the relevant tender (e.g. “security guard services”). Fifteen are *paraphrases* that use a vendor’s natural wording (e.g. “cellular handsets for sales staff” for tenders on smartphones and feature phones; “rods for reinforced concrete” for MS deformed bars; “photovoltaic panels” for a rooftop solar system). **Systems compared.** (a) *Keyword*, the platform’s lexical arm alone (PostgreSQL `plainto_tsquery` + `ts_rank_cd` on the weighted tsvector), which represents the search that e-GP and the private aggregators offer; (b) *Semantic*, the MiniLM + pgvector arm alone with the 0.8 ceiling; (c) *Hybrid*, the deployed RRF fusion

(Equation (4.1)). **Metrics.** Hit@5, Recall@5, mean reciprocal rank (MRR), and the number of queries returning no results at all.

Table 5.2: E2: retrieval quality on the 25-query benchmark

Query set	System	Hit@5	Recall@5	MRR	Zero-result queries
Lexical (10)	Keyword	0.80	0.75	0.80	2
	Semantic	1.00	1.00	0.93	0
	Hybrid	1.00	1.00	1.00	0
Paraphrase (15)	Keyword	0.33	0.33	0.33	10
	Semantic	1.00	1.00	1.00	0
	Hybrid	1.00	1.00	1.00	0
All (25)	Keyword	0.52	0.50	0.52	12
	Semantic	1.00	1.00	0.97	0
	Hybrid	1.00	1.00	1.00	0

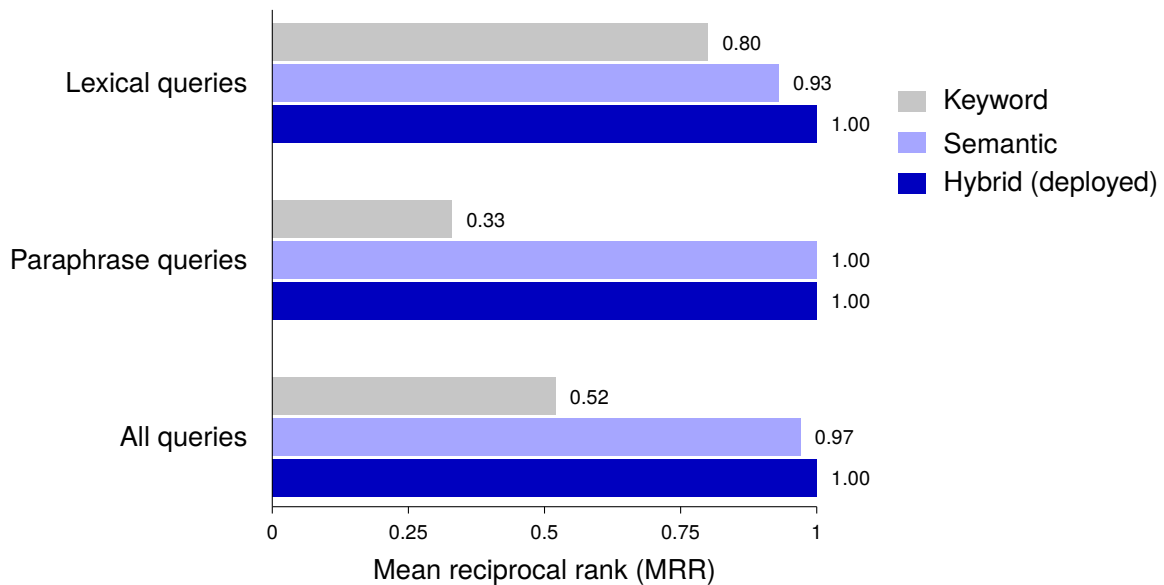


Figure 5.1: E2: MRR by system and query type

Result. Keyword search, the approach used by existing portals, returned *nothing at all* for 12 of the 25 queries, including 10 of the 15 paraphrases. Its failures are exactly the vocabulary mismatches that cost vendors opportunities in practice: “cellular handsets” vs. “smartphones”, “rods” vs. “deformed bars”, “asphalt” vs. “bitumen”, “rice seed” vs. “paddy seeds”. The implicit AND of `plainto_tsquery` also made it fail two lexical queries (“water supply pipeline *repair*” and “medical equipment *for ICU*”) because one query word was absent from the tender. Hybrid search found a relevant tender in the top five for every query and ranked one *first* every time (MRR 1.00 vs. 0.52, Table 5.2 and Fig. 5.1). Fusion also improves on semantic search alone: for “stationery items” the vector arm ranked two loosely related tenders above the exact match (MRR 0.33), and RRF restored the exact match to rank 1 because the lexical arm agreed. This is the complementarity reported in the literature [18], confirmed here on procurement text.

Threats to validity. The corpus is small and team-curated, so absolute numbers will be lower on a large, noisy production corpus. The *relative* result is robust, however: keyword search cannot match a synonym it has never seen, whatever the corpus size.

5.1.3 E3: Similarity ceiling for off-topic queries (RQ3)

Five queries unrelated to procurement were run against the same 48 tenders.

Table 5.3: E3: semantic matches returned for off-topic queries (ceiling $\delta < 0.8$)

Query	Nearest distance	Results with ceiling	Results without ceiling
football match tickets	0.690	1	48
birthday cake recipe	0.801	0	48
quantum chromodynamics lecture	0.918	0	48
guitar lessons	0.768	2	48
holiday in Maldives	0.749	1	48
Total		4	240

Result. The ceiling cut irrelevant semantic results from 240 to 4 (a 98% reduction, Table 5.3), while all 25 benchmark queries in E2 kept their relevant results. The design refinement is therefore justified.

5.1.4 E4: Latency on CPU (RQ4)

Table 5.4: E4: measured latency (20 repetitions, CPU only)

Operation	Median	p95
Embed short query/title (384-d)	7.0 ms	7.8 ms
Embed 2,000-character tender description	27.9 ms	31.6 ms
End-to-end hybrid search (embed + 2 SQL + RRF)	8.0 ms	12.5 ms (max)
PDF text extraction (2–3 page notice)	99–121 ms	
One-time model load at service start	≈ 10 s	

Result. Search is about *two orders of magnitude* faster than the 1 s target (Table 5.4), and embedding a page costs 28 ms against the 200 ms planning estimate. This confirms that the no-GPU infrastructure plan is sound. The model loads once and is cached, so it adds no per-request cost.

5.1.5 E5: Financial scoring behaviour (RQ5)

A tender with budget BDT 10–15 lakh (midpoint 12.5 lakh) received five bids. Table 5.5 shows the scores produced by the production function.

Table 5.5: E5: financial scoring and outlier flag on a worked example

Bid (BDT)	F	Low outlier	Note generated
11,00,000	84.80	no	12% under the budget midpoint
13,00,000	76.00	no	4% above the midpoint, within ceiling
14,50,000	64.00	no	16% above the midpoint, within ceiling
16,00,000	43.33	no	7% over the budget ceiling
4,80,000	100.00	yes	62% under midpoint; flagged to verify feasibility

Result. Scores decrease monotonically with price, bids over the ceiling are penalised in proportion to the overrun, and the 4.8-lakh bid is the only one flagged (median 13 lakh, fence 6.5 lakh). The earlier quartile-based fence would have been pulled down by the legitimately cheaper 11-lakh bid. The buyer thus sees a plain-language warning rather than an opaque penalty, which keeps the final judgement with the human.

Synthesis

The experiments support the core technical claims. Real e-GP notices can be imported with 90% of fields exact (RQ1). Hybrid search removes the zero-result failures of keyword search and ranks the right tender first on the benchmark (RQ2). The similarity ceiling suppresses 98% of off-topic noise (RQ3). CPU-only inference is comfortably interactive (RQ4). The evaluation scoring behaves transparently and as designed (RQ5).

5.2 Performance Evaluation

5.2.1 Automated verification

Setup. pytest with pytest-asyncio and httpx against the FastAPI application, with the database and external services replaced by dependency overrides and mocks. The suite runs in CI on every change. **Acceptance criterion.** All tests pass on main before deployment (enforced by the CI-gated CD pipeline).

5.2.2 Evaluation against objectives

5.2.3 Evaluation against functional requirements

Table 5.8 gives the status of each requirement in the 24 February baseline. Of the 21 *Must* requirements, 15 are fully met, 5 are met in their core with a stated refinement pending, and

Table 5.6: Automated test suite by area (344 passed, 2 skipped live-database tests, 0 failed)

Area	Tests	What is verified
Tenders	85	Create/draft/publish/edit/withdraw/delete, lifecycle rules, visibility filter, PDF import job, auto-close
Auth	40	Login, invitation registration, JWT handling, password-reset token lifecycle
Bids	38	Submission with documents, update/withdraw rules, comparison, acceptance and award
Messaging	37	DMs, groups, tender rooms, membership and admin rules
Core / RBAC	22	Role dependencies, permission denials, database initialisation
Payments	19	Balances, packages, pricing, purchase and ledger consistency
Notifications	15	Creation, unread counts, read/delete
Admin	14	Admin-only guards, verification, pricing and package management
Audit	14	Hash-chain construction, tamper and deletion detection , Merkle archives
FR-08/09/10 + reviews	11	Packaged lots, recommendations, comparison, mutual reviews
Organisations, users	20	Sign-up, invitations, roles, enlistment, profile, documents
Contracts	9	Completion and review rules
Search	8	Hybrid query, visibility scoping, lexical fallback
Middleware, storage	13	Audit/request-logging middleware, file storage
Total	346	344 passed; 2 skipped (require a live database)

Table 5.7: Achievement of project objectives

Objective	Evidence	Status
O1 Identity & access	KYC sign-up with admin verification; e-mail invitations; 4 roles; RBAC engine; visibility filter shared by list/detail/search; 22 RBAC/core + 14 admin tests	Met
O2 Tender lifecycle	Draft/publish/edit/withdraw/delete, packaged lots, required documents, hourly auto-close, PDF import (E1); 85 tests	Met
O3 Bidding & award	Remote bid with documents, update/withdraw, comparison, award, contract completion, mutual reviews; 38 + 9 + 11 tests	Met
O4 Intelligence	(a) 27/30 fields (E1); (b) MRR 1.00 vs. 0.52 (E2); (c) explainable recommendations (Equation (4.2)); (d) 4-component evaluation (E5)	Met
O5 Communication	Encrypted DMs, groups and tender rooms with owner-managed membership; real-time notifications; 37 + 15 tests	Met
O6 Accountability	All state-changing requests audited via outbox; hash chain with on-demand verification; tamper tests pass	Met
O7 Economics	Token balance, packages, admin pricing, append-only ledger; 19 tests	Met (sandbox gateway)
O8 Engineering quality	344 passing tests, CI on every change, CI-gated Docker deployment to EC2	Met

one (FR-19, bid-bond handling) is deferred for the reason given in Section 5.3. Both *Should* requirements are met. Several requirements are **exceeded**: FR-10 adds AI evaluation, FR-14 reviews are mutual rather than one-way, FR-22 adds cryptographic tamper evidence, and FR-08 adds PDF import.

Table 5.8: Status of functional requirements

FR	Implementation evidence	Status
01	About, policies, legal, news, events, help, support pages	Met
02	/public/active, public detail with summary-only fields	Met
03	Hybrid RRF search on the vendor dashboard; lexical fallback	Met, exceeded (E2)
04	Master account with NID + KYC files, admin verification, member sign-up by invitation	Core met; e-mail/SMS OTP pending
05	Login/logout, refresh, e-mail password reset, password change	Core met; 2FA pending
06	E-mail invitations, accept, role assignment, unique organisation join code	Met
07	DMs, groups, owner-managed tender rooms, AES-GCM, real-time notifications	Met
08	Full lifecycle, packaged lots, required docs, auto-close, PDF import	Met, exceeded
09	Four-factor recommendations with reasons	Met
10	Side-by-side comparison + AI evaluation runs	Met, exceeded
11	Accept bid $\Rightarrow$ award, others rejected, all notified	Core met; formal NOA acceptance step pending
12	Restricted tenders visible only to enlisted vendors, enforced in SQL	Met
13	Contract record and completion confirmation	Met
14	Mutual buyer $\leftrightarrow$ seller reviews feeding reputation	Met, exceeded
15	Seller dashboard with live stats and semantic search	Met
16	Bid with proposal and buyer-required documents	Core met; bid-bond payment pending
17	Status lifecycle with timestamps; “My bids” view	Met
18	Balance, packages, purchase, ledger	Met (sandbox)
19	Bid-bond holding and refund	Deferred

FR	Implementation evidence	Status
20	Admin-configurable publish/bid cost, token price, packages	Met
21	Verification queue, pricing and packages, admin guards	Met
22	Outbox + hash chain + verification + Merkle archives	Met, exceeded
23	Dispute management (tentative)	Future work
24	Reporting and analytics (tentative)	Future work
25	Gateway-shaped purchase flow with transaction and validation IDs	Sandbox; live merchant pending
26	External identity APIs (tentative)	Replaced by manual verification

5.2.4 Non-functional requirements

- **Security (NFR-1, 2):** bcrypt, HttpOnly JWT cookies, RBAC on every protected endpoint (verified by permission-denial tests), AES-GCM messages, and competitors unable to read each other's bids.
- **Auditability (NFR-3):** the tamper tests alter a record's payload and remove a record from the sequence, and confirm that verification reports the exact break.
- **Performance (NFR-4):** 8 ms median search, 28 ms per embedding; PDF import and evaluation run asynchronously.
- **Availability (NFR-5):** search degrades to full-text when the ML service is unavailable (tested); LLM failures degrade to the rule parser or *needs_review*.
- **Scalability (NFR-6):** the API and ML containers are stateless and can be replicated; the database supports read replicas.
- **Maintainability (NFR-8):** domain modules, 346 tests, CI/CD.
- **Usability (NFR-7):** a consistent enterprise design system across all dashboards (Figures 4.8 and 4.9), reviewed with the industry mentor. A formal usability study is future work (Section 7.1).

5.3 Deviations and Design Revisions

D1: “Local AI only” → local embeddings with an optional hosted LLM. *Cause:* the inception plan kept all NLP on our own server. Rule-based extraction alone, however, cannot generalise across notice layouts, and running an LLM locally needs a GPU that the budget excludes (Section 2.4). *Revision:* embeddings, which touch every tender and query, stay local. Only the tender or bid text under evaluation is sent to a hosted LLM (Groq), no KYC documents, personal data or other organisations’ bids are sent, and a deterministic parser (E1) serves as the fallback. The provider is configurable, so a self-hosted model can replace it without code changes once hardware allows.

D2: NOA acceptance and bid bonds (FR-11, FR-16, FR-19). *Cause:* holding bid-bond money on behalf of third parties requires a payment-service arrangement (escrow or a merchant account with split settlement), which a student project cannot legally operate. *Revision:* the award is recorded atomically at bid acceptance, with notifications to every bidder. The schema already contains `bid_securities` and a `WalletHold` security type, so the NOA-acceptance step and bond handling can be added without a data-model change once a licensed partner is in place.

D3: Live payment gateway (FR-25). *Cause:* SSLCommerz production credentials require a registered business. *Revision:* the purchase service follows the gateway’s contract (transaction ID, validation ID, ledger entry in one transaction), so switching to the live gateway only requires the HTTP call and callback endpoint.

D4: OTP and 2FA (FR-04, FR-05). *Cause:* SMS OTP needs a paid BTRC-compliant SMS aggregator. *Revision:* identity assurance currently rests on administrator KYC review and e-mail-based flows; the TOTP library (`pyotp`) is already a dependency for adding authenticator-app 2FA.

D5: Tentative requirements (FR-23, 24, 26). These were marked “not sure” in the baseline. Endpoint stubs exist, and their implementation is planned as future work (Section 7.1).

Revisions made to reach compliance. The design revisions of Section 4.2 (FastAPI consolidation, modular monolith, asynchronous PDF import, 384-d vectors, median outlier fence, similarity ceiling) were each made in response to a measured or prototyped deficiency, and each is verified by the tests and experiments above.

Chapter 6

Project Management, Teamwork, and Impact

6.1 Project Management and Finance

6.1.1 Planning and Risk Management

The project ran in two phases. **Phase 1 (analysis and design, February–March 2026)** had weekly milestones: form the focus group and finalise the questionnaire (10–13 Feb); discuss requirements and draft the schema and ERD (14–20 Feb); design modules and data flow (21–27 Feb); produce use-case, interaction and data-flow diagrams (28 Feb–6 Mar); and build a first prototype of the core feature (7–13 Mar). **Phase 2 (implementation, April–September 2026)** delivered the modules listed in the inception report: presentation layer, user management, tender management, reputation, search, security, bidding, chat, and payment. Figure 6.1 shows the schedule as executed.

Where progress differed from plan. Phase 1 kept to its weekly schedule. The main change in Phase 2 was the back-end language. After the April prototype, the team decided to consolidate on Python (Section 4.2) and re-based the back end in June, so feature development was concentrated in July–September, the most active period (118 of 134 commits). To keep that period on track, the team ran an internal completion audit against the 26-requirement baseline at the end of August. It produced a prioritised tracker of work items (P0 to P3). The following sprint delivered the RBAC engine, semantic search, smart bid evaluation and password reset, all merged between 29 August and 1 September, followed by CI/CD and deployment.

6.1.2 Budgeting and Resource Identification

People. Five developers for about eight months. At an estimated 10 hours per member per week over 32 weeks (about 1,600 person-hours) and a junior-developer rate of BDT 500 per hour, the notional labour value is about BDT 8 lakh. This value was contributed by the team, not paid.

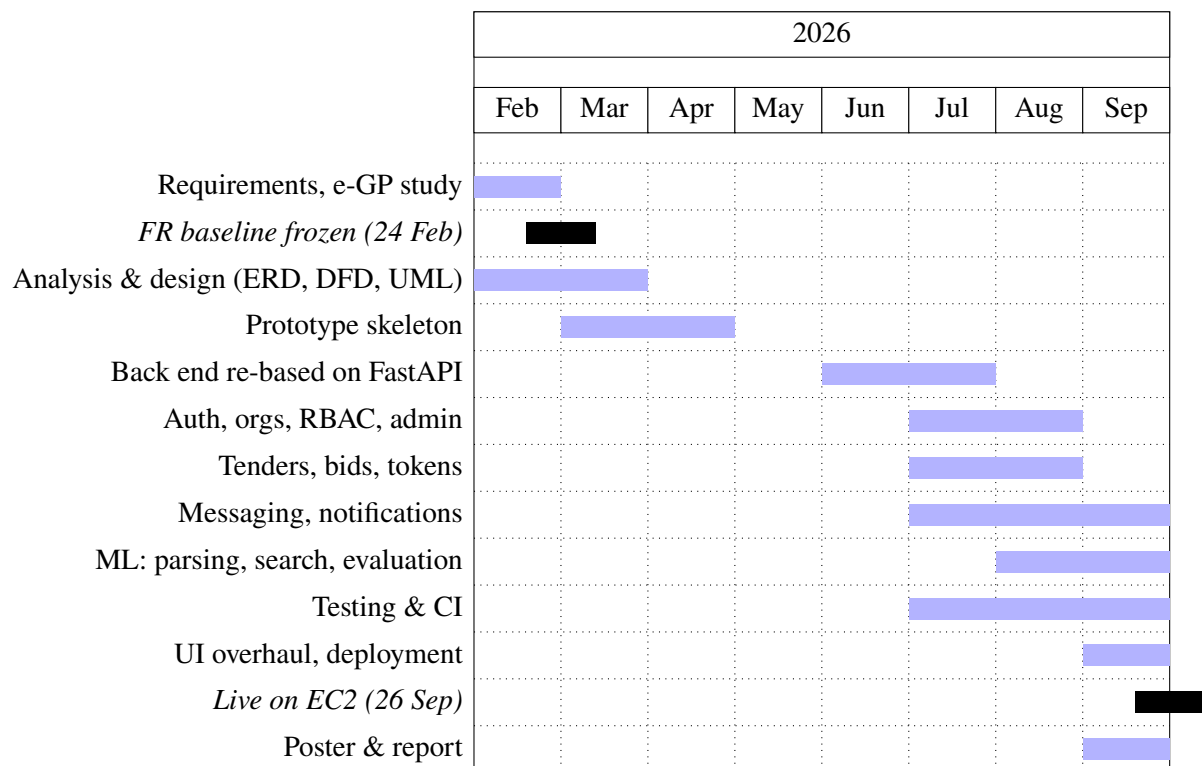


Figure 6.1: Project schedule as executed (February–September 2026)

Table 6.1: Major risks and mitigation

Risk	Likelihood	Impact	Mitigation (and outcome)
ML features too slow without a GPU	Med	High	Chose a 384-d CPU model; measured early; asynchronous jobs. <i>Outcome:</i> 8 ms search (E4).
LLM unavailability or malformed output	High	Med	Structured output, 3 retries, rule-based fallback, <i>needs_review</i> state.
Scope creep beyond one session	High	High	MoSCoW priorities; “not sure” items deferred; tracker with P0–P3.
Schema drift between branches	Med	High	Numbered SQL migrations; a live-schema test; one owner for the data model.
Security flaw exposing bids or KYC data	Med	High	Central RBAC, shared visibility filter, admin guards, audit log, tests for denials.
Payment and escrow regulation	High	Med	Sandbox gateway; bid-bond data model ready for a licensed partner.
Build out-of-memory on the small host	Med	Med	Sequential image builds in CD (COMPOSE_PARALLEL_LIMIT=1).
Uneven availability of team members during exams	High	Med	Module ownership with a backup reviewer; asynchronous coordination through GitHub and a shared document.

Development resources. Team-owned laptops; open-source software throughout (Table 6.7); GitHub and GitHub Actions free tiers; managed PostgreSQL, the EC2 host and the LLM API at development-scale usage. No hardware was purchased.

Production budget. Table 6.2 gives the itemised monthly budget for a production deployment for 1,000 users, priced from the BDCCL catalogue so that data stays inside Bangladesh.

Table 6.2: Itemised monthly production budget (BDCCL catalogue)

Item	Service code	Quantity	Unit (BDT)	Monthly (BDT)
Compute (4 OCPU)	BIS 01	4	3,597	14,388
Memory	BIS 08	16 GB	130	2,080
Block storage (OS, DB)	BDS03	40 GB	18	720
Object storage (documents)	BDS05	50 GB	12	600
Outbound bandwidth	BDT21	100 GB	5	500
Public IP	—	1	included	0
Linux OS (no licence fee)	—	—	0	0
Domain name (US\$11.5/yr)	—	—	—	≈120
Total				≈18,408

Usage-dependent items are not included in the table: the payment gateway’s per-transaction fee (passed through in token prices), transactional e-mail and SMS (for OTP, once added), and LLM usage, which is bounded because it runs only on PDF import and on explicit evaluation requests. The CPU-only design avoids a GPU instance of about BDT 255,000 per month, a 93% saving.

6.1.3 Economic Analysis and Financial Prospecting

Revenue model. Revenue comes from usage-based tokens. With the default pricing (BDT 1 per token, 50 tokens to publish, 20 to bid), a tender that attracts n bids earns $R(n) = 50 + 20n$ taka. With an average of six bids this is BDT 170 per tender cycle; with the largest package discount of 25%, it is about BDT 128.

Break-even. Against a monthly operating cost of about BDT 18,400:

For comparison, e-GP invited 20,000 tenders in a single month in 2025 [1]. The platform therefore breaks even at a volume below 1% of the public sector’s monthly peak, with a single commodity server.

Value to users. For a *buyer*, the fee of about BDT 50 to publish a tender replaces a national newspaper advertisement, which typically costs orders of magnitude more, plus the staff time spent distributing documents and receiving and logging sealed bids. PDF import and AI evaluation

Table 6.3: Monthly break-even and revenue scenarios (6 bids per tender)

Scenario	Tenders / month	Revenue (BDT)	Margin (BDT)
Break-even (list price)	109	18,530	≈0
Break-even (max. discount)	145	18,488	≈0
Growth (1% of e-GP peak volume)	200	34,000	15,600
Expansion	500	85,000	66,600
Scale (10% of e-GP peak volume)	2,000	340,000	321,600*

*Before scaling infrastructure; the stateless design adds nodes at the same unit cost.

further reduce preparation and evaluation effort. For a *vendor*, a BDT 20 bid replaces the purchase of tender documents, printing and travel to submit. For a vendor outside Dhaka this can be a full day's travel per tender. Because the pricing is usage-based rather than a subscription, SMEs pay only when they bid. Wider participation also tends to lower prices for buyers (Section 2.1), a gain that dwarfs the platform fee.

Interpretation. Development cost is sunk (team effort), and operating costs are low and nearly fixed up to about 1,000 active users. The marginal cost of a tender is close to zero (a 28 ms embedding and a few rows). Revenue scales linearly with activity, so the business becomes profitable at modest adoption, and the administrator can tune prices and packages without code changes.

6.1.4 Sustainability in Business and Commercialisation

Path to market. Our industry partner, Advanced Information Systems, runs AllTender.com, which already has an audience of vendors who follow tender notices. The most practical path is to position ProcureNext as the *transaction layer* behind that audience: buyers whose notices already appear on AllTender can be invited to run the tender itself on ProcureNext. The planned steps are: (1) a pilot with a small group of the partner's buyer clients; (2) go-live of the SSLCommerz merchant account and SMS OTP; (3) a bank or escrow partner for bid bonds (FR-19); (4) migration of hosting to BDCCL for data residency; (5) Bangla localisation to reach more of the market.

Who maintains it. After the course, the natural owner is the industry partner's engineering team. The handover package consists of the repository with full history, 346 automated tests, CI/CD pipelines, Docker configuration, SQL migrations, and the analysis documents (requirements, ERD, DFD, API list). Team members remain available for knowledge transfer. The modular structure lets new developers own one module without having to learn the whole codebase.

Why it stays viable. Low fixed costs, usage-based revenue that covers them at small volume, an existing distribution channel through the partner, and open-source dependencies without licence fees.

6.2 Individual and Teamwork

6.2.1 Participation and Contribution

Roles follow the team's final allocation (as presented at Capstone Fair 2026). Contributions are evidenced by the project documents and by repository history.

Table 6.4: Roles and contributions of team members

Member	Role and specific contributions
Tanvin Islam Abir (2105020)	Database Design & RBAC. Wrote the structured study of the e-GP public surface that seeded the requirements; designed the database schema, the ERD and the detailed data-flow diagrams; implemented organisation invitations and role assignment; built the RBAC permissions engine (<code>core/permissions.py</code>); designed the packaged-lot and mutual-review schema extensions; wrote the test suites for FR-08/09/10, contracts and mutual reviews.
Md. Mahmud Hasan (2105027)	ML Engineer. Set up the initial repository and containerised development environment; built the ML service (tender PDF parsing, the 384-d embedding pipeline, and embedding generation at tender creation); implemented the hybrid semantic search with RRF on the vendor dashboard; designed and implemented Smart Bid Evaluation (the four-component scoring, evaluation runs, and the asynchronous evaluation task). Coordinated the research questions for requirement engineering.
Tawhid Umar (2105028)	Full-Stack Developer & DevOps. Built the first back-end skeleton and then the FastAPI back end; led the tender module (lifecycle, drafts, edit/withdraw/delete, auto-close) and the related buyer workspace; led the enterprise UI overhaul across dashboards and tender workflows; wrote the Docker Compose configurations, the CI pipeline and the CI-gated CD workflow; deployed and operates the production system on EC2.

Member	Role and specific contributions
Fatin Sadab (2105029)	Full-Stack Developer. Authored the functional and non-functional requirements, use-case identification and module definitions, and the formal use-case and interaction diagrams; implemented the administrator approval flow for master accounts with document upload and temporary links; implemented authentication; built real-time messaging (intra-company DMs and groups, inter-company tender group chats) and the notification system; wrote the auth, admin and messaging tests; led the removal of the Node.js back end.
Safayat Saif (2105030)	Full-Stack Developer. Wrote the market study, solution strategy and technology-stack justification of the inception report; implemented the token economy (balances, pricing, packages, ledger) with its test suites; e-mail delivery of member invitations over SMTP; password reset; tender and bid fixes during integration; production fixes at deployment (tender creation and the cookie Secure flag over HTTP).

Every member took part in weekly discussions and supervisor meetings. Key decisions were taken collectively: the FR baseline, the monolith choice, the FastAPI consolidation, and the prioritisation after the August audit. Module owners delivered their modules for integration before the final deployment milestone on 26 September.

6.2.2 Collaboration and Conflict Resolution

The team collaborated through a shared living requirements document (which recorded meeting notes, requirement versions and design drafts), GitHub feature branches and pull requests, and messaging groups for day-to-day coordination. Merges were spread across all five members, so no one person was a bottleneck. Three conflicts are worth recording.

- **Architecture.** The analysis-phase documents argued for microservices, while the inception report argued for a modular monolith. Instead of settling this by opinion, we prototyped the most cross-cutting flow (accepting a bid) and discussed the result with our supervisors. The team agreed on a monolith with a separate ML service, which kept the useful part of each position.
- **Language.** An early Express prototype conflicted with the Python ML work. The trade-off (two runtimes vs. one) was laid out, and the team consolidated on FastAPI. The member who had written much of the Node code led its removal.

- **Parallel schema changes.** A schema rewrite on one branch dropped a table that another feature depended on. The team adopted numbered migration scripts and a live-schema check, and the table was restored in a follow-up commit.

The atmosphere stayed constructive because disagreements were argued with evidence (a prototype, a test, a measurement) rather than seniority.

6.2.3 Leadership and Direction

The team chose *shared leadership*. Instead of a single project lead, leadership rotated with the phase and the domain. The members responsible for requirements (Fatin Sadab, Tanvin Islam Abir) set direction in Phase 1; module owners led their areas in Phase 2; the ML engineer (Md. Mahmud Hasan) led decisions on intelligent features; and the DevOps owner (Tawhid Umar) led integration and release. Direction toward the goals came from three artefacts everyone could see: the frozen FR baseline, the Gantt milestones, and, from August, the prioritised tracker. Members' views were collected in the weekly meeting before decisions were recorded. After setbacks, notably the language change and the slower spring, the team re-planned openly. It re-sequenced the modules, ran the completion audit, and focused on the highest priorities, and it was this re-planning that produced the late-August delivery sprint. Motivation came from visible progress (each merged feature was demoed to the team) and from the goal of a live system at the Capstone Fair.

6.2.4 Multidisciplinary Engagement

The project required knowledge well outside computer science:

- **Procurement practice and law:** the team studied e-GP's workflows and vocabulary, the PPA 2006/PPR 2008, bid securities and notices of award, and adapted them for private use.
- **Industry engagement:** our mentor at Advanced Information Systems (AllTender.com) reviewed the requirements. The organisation-centric account model, enlisted-vendor tenders, owner-supervised inter-company chat and offline contract signing all came from this engagement.
- **Business and finance:** token pricing, package design, infrastructure costing from the BDCCL catalogue, and break-even analysis.
- **Regulatory and compliance:** Bangladeshi KYC documents (trade licence, TIN, VAT/BIN, NID), data protection, and payment-industry scope.
- **Human factors:** designing for non-technical procurement officers, and presenting AI output as explained assistance rather than a verdict.

6.3 Impact on Society

Table 6.5: Societal impacts and design responses

Dimension	Impact (who, how)	Design response
Safety	(+) Vendors no longer have to deliver bids in person to a tender box, where “muscle power” intimidation occurs.	Fully remote submission; bids invisible to competitors; no physical point of interference.
Health	(+) Fewer journeys and crowded submission counters. (–) Screen-based work for procurement staff.	Remote workflow; clear dashboards that reduce time spent searching.
Economic fairness	(+) SMEs and vendors outside Dhaka gain access to private tenders.	Semantic search that does not require knowing the buyer’s exact wording; low per-bid pricing; free browsing.
Transparency	(+) Reduced favouritism and disputes; an evidential record.	Hash-chained audit log; timestamped bid status; mutual reviews.
Legal	(+) Records admissible under the ICT Act. (–) Risk of misuse by fraudulent firms.	KYC with manual admin verification; contracts signed under the Contract Act; audit trail supports dispute resolution.
Cultural	(–) English-only interface and documents exclude some users; a relationship-based procurement culture may resist a formal process.	Plain, consistent UI; e-GP-familiar vocabulary; Bangla localisation planned; buyers keep discretion over awards and enlisted vendors.
Employment	(–) Less work for intermediaries who hand-deliver bids or place newspaper ads.	New roles for procurement officers as platform users; the platform does not remove human evaluators.
Privacy	(–) Concentration of business and personal data.	Least privilege, encrypted messages, no data sale, KYC documents visible only to administrators.
AI influence	(–) Automated scores could unduly steer awards.	Explained, weighted, reproducible scores; <i>needs_review</i> state; the award is always a human action.

6.4 Environmental Impact and Life Cycle Sustainability

Development. The team used its own laptops and free CI runners, and bought no new hardware. CI runs are short (about 3 s of backend tests plus a front-end build).

Deployment and operation. The whole platform runs on one virtual machine without a GPU. Choosing a small CPU model over GPU inference avoids a large, continuously powered accelerator. Embeddings are computed *once* per tender at publish time and reused for every search, and the model is loaded once per process. The LLM is used only on explicit import or evaluation actions.

Paper and travel avoided. As an illustrative estimate, a tender with eight bids of 150 pages each consumes 1,200 printed sheets (about 6 kg of paper at 5 g per A4 sheet) plus the buyer's copies of the tender document. On ProcureNext this becomes a few megabytes of storage, and each vendor saves at least one round trip to deliver the bid.

Retirement of the system and its data. Because the platform is containerised on rented cloud infrastructure, retirement produces no e-waste from the project itself. The data needs a defined end of life. Tender and audit records should be kept for a fixed period to support disputes (seven years is common practice for procurement records), then exported to the organisations concerned and securely deleted. The Merkle-root archives allow a compact integrity proof to be kept even after bulk data is removed.

Table 6.6: Negative environmental impacts and mitigation

Negative impact	Mitigation
Data-centre electricity for 24/7 hosting	Single right-sized VM; no GPU; idle ML container uses almost no CPU; scale out only on demand.
Energy of hosted LLM calls	Called only on explicit actions; deterministic parser first where sufficient; configurable provider to move to efficient local models.
Unbounded growth of stored documents	2 MB-per-document sizing; object-storage lifecycle rules to move old files to cold storage; retention schedule with deletion.
Repeated container builds in CI/CD	Build only after tests pass on main; layer caching; sequential builds.
Rebound effect (more tenders because they are cheap)	Token cost for publishing and bidding discourages spam tenders and bids.

6.5 Ethics

6.5.1 Equity and Inclusivity

- **Access for small and remote vendors:** browsing and search are free, bidding costs BDT 20, and submission needs only a browser, which levels the field against large local firms.
- **Fair discovery:** semantic search finds tenders even when a vendor's vocabulary differs from the buyer's (E2), which helps less experienced vendors who do not know the buyer's standard terms.
- **Fair recommendations:** new vendors receive a neutral default rating (3/5) rather than zero, so a lack of history does not exclude them. All factors are shown, so buyers can see why a vendor is suggested and judge fairness themselves.
- **Fair evaluation:** low bids are never silently penalised; they are flagged for human review with an explanation.
- **Accessibility:** the responsive layout works on phones; components follow WCAG practice (labels, contrast, keyboard use). A full WCAG audit and Bangla support are planned, since the English-only interface is the main remaining exclusion.

6.5.2 Accountability and Personal Responsibility

The platform makes *actions* accountable: every state change is attributed to a specific person in a specific organisation and sealed in the hash chain, and administrators' own actions are audited too. Responsibility for *decisions* stays with people: the buyer awards, and AI output is labelled as assistance, with its model, prompt version and weights stored. For the *system's* behaviour, the team is responsible during the course, with each module owner (Table 6.4) answerable for their module's correctness and tests; after handover, responsibility passes to the operator (Section 6.1.4). Members took responsibility for their own work in practice. Fixes found at deployment were made promptly by the owning member, and each member's contributions can be traced through the commit history.

6.5.3 Proper Use of Intellectual Property

All sources are cited in the References. Table 6.7 lists the main third-party components and their licences. All are open source and permit commercial use. Real e-GP notices are public government documents and were used only as test inputs. The UI design is original to the team, and the screenshots are of our own system.

Table 6.7: Third-party software, models and services

Component	Licence	Use
FastAPI, Pydantic, python-jose, LangChain	MIT	API framework, validation, JWT, LLM orchestration
Next.js, React, Tailwind CSS	MIT	Front end
PostgreSQL, pgvector	PostgreSQL Licence	Database, vector search
asyncpg, bcrypt, sentence-transformers	Apache-2.0	DB driver, hashing, embeddings
all-MiniLM-L6-v2 model	Apache-2.0	Sentence embeddings
cryptography	Apache-2.0 / BSD	AES-GCM
Celery, pypdf	BSD-3-Clause	Task queue, PDF text
Redis 7	BSD-3 (≤ 7.2) / RSALv2–SSPL (≥ 7.4)	Broker; unmodified internal use permitted
Groq API	Commercial terms of service	Optional LLM inference
Docker Engine, GitHub Actions	Apache-2.0 / service terms	Containers, CI/CD

Generative AI disclosure. The team used generative AI assistants (i) during design, as one input among several (one architecture-guideline draft in our working document was AI-generated and then critically revised); (ii) during development, for code suggestions that members reviewed, tested and frequently rewrote; and (iii) for help in drafting and editing this report. All technical claims were checked against the code, and every experiment in Chapter 5 was run by executing the project’s own code. The team takes full responsibility for the content.

6.5.4 Professionalism and Ethical Codes

The ACM Code of Ethics [44], the IEEE Code of Ethics [45] and the IEB code of conduct all put the public interest first. These codes guided specific decisions:

- **Avoid harm (ACM 1.2) and honesty about AI (ACM 1.3):** AI scores are advisory, explained and reproducible; the system reports *needs_review* rather than fabricating a score when the model fails.
- **Fairness and non-discrimination (ACM 1.4; IEEE I.7–8):** additive, transparent recommendation factors that exclude protected attributes; neutral default ratings for newcomers.
- **Respect privacy and confidentiality (ACM 1.6–1.7):** bids sealed from competitors, encrypted messages, minimal data sent to external AI services (Section 5.3, D1).
- **Quality and security (ACM 2.1, 2.9):** 346 automated tests, CI gate before deployment, OWASP-aligned controls.
- **Honest reporting (IEEE I.5):** this report states the curated nature of the search benchmark and lists the deferred requirements instead of claiming full completion.
- **Public interest in procurement:** the platform’s purpose, fair and safe access to private-sector contracts, is itself a response to the engineer’s duty to society.

Chapter 7

Conclusion

7.1 Conclusion and Future Work

ProcureNext set out to give Bangladesh's private sector what e-GP gave the public sector, a digital, remote and recorded tender process, and to go beyond it with intelligence and flexibility. Against the objectives of Section 1.4:

- **O1–O3** are met. Organisations onboard with verified KYC documents and manage their own members under four roles. Buyers run the full tender lifecycle, including multi-lot packages, restricted tenders and automatic closing. Vendors bid remotely with documents, and every award leads to contract completion and mutual reviews.
- **O4** is met and is the project's strongest technical result. Real e-GP notices import with 27 of 30 fields exact. Hybrid search ranked a relevant tender first for all 25 benchmark queries, while keyword search, the approach of existing portals, returned nothing for 12. Recommendations and bid evaluations are explained factor by factor.
- **O5–O6** are met: encrypted, owner-supervised messaging with real-time notifications, and a hash-chained audit log whose tamper detection is tested.
- **O7** is met in sandbox mode. The token economy breaks even at under 1% of e-GP's monthly peak volume.
- **O8** is met: 344 passing tests, CI on every change, and CI-gated deployment of a live system on AWS EC2.

Limitations. The retrieval benchmark is small and curated. Bid-bond handling, the formal NOA acceptance step and the live payment gateway await a licensed financial partner. OTP and 2FA are not yet enabled. The interface and document processing are English-only. LLM-based features depend on a hosted provider, with a deterministic fallback.

Future work.

1. **Financial completion:** go live with SSLCommerz; implement bid bonds and the NOA acceptance step with an escrow or bank partner (FR-11, FR-19, FR-25).
2. **Identity assurance:** SMS OTP and authenticator-app 2FA; integration with NID and TIN verification services where permitted (FR-04, FR-05, FR-26).
3. **Retrieval at scale:** build a relevance-labelled benchmark from pilot usage logs; add an HNSW index and evaluate on thousands of real tenders; learn fusion weights from clicks.
4. **Language:** Bangla UI, Bangla/English cross-lingual embeddings, and OCR for scanned notices.
5. **Self-hosted LLM:** a quantised open model for extraction and rubric scoring once hardware allows, which would remove the external dependency.
6. **Governance features:** dispute management and exportable analytics (FR-23, FR-24); external anchoring of Merkle roots.
7. **User validation:** a formal usability study with procurement officers and SME vendors during the partner pilot, measuring onboarding time, task success and trust in AI explanations.

7.2 Continuous Engagement and Self-directed Learning

Most of the technology in ProcureNext was new to the team at the start of the session. It was learned independently from primary documentation and current literature:

- **Frameworks at their current versions:** FastAPI's dependency injection and lifespan model; Next.js 16 with React 19 and its server-side route handlers (used for the HttpOnly-cookie proxy); Tailwind CSS 4.
- **Vector search:** the pgvector documentation (distance operators, index types), the Sentence-BERT and MiniLM papers, the MTEB leaderboard for model selection, and the original RRF paper, whose $k = 60$ we adopted.
- **LLM engineering:** structured output with LangChain agents, retry and fallback design, prompt versioning, and the LLM-as-judge literature on the biases that led us to make scores advisory.
- **Security engineering:** OWASP ASVS, NIST SP 800-63B for password reset, NIST SP 800-38D for AES-GCM nonce handling, and the tamper-evident logging literature (Schneier–Kelsey, Crosby–Wallach), which we implemented from first principles together with a transactional-outbox pattern.

- **DevOps:** GitHub Actions `workflow_run` chaining for CI-gated deployment, Docker multi-stage builds, and memory-aware sequential builds on a small host.
- **Domain knowledge:** the Public Procurement Act and Rules, e-GP procedures and notice formats, BPPA's published statistics, and private-sector practice learned from our industry mentor.

The team also practised self-assessment. The August completion audit, which checked our own work against our own requirement baseline, is the clearest example. It turned a vague sense of “what’s left” into a prioritised plan and set the habit of verifying claims with tests and measurements. That habit is also the basis of this report.

Bibliography

- [1] The Business Standard, “e-GP system sees record surge in tenders, registrations: BPPA,” <https://www.tbsnews.net/bangladesh/e-gp-system-sees-record-surge-tenders-registrations-bppa-1305721>, Dec. 2025, accessed: 29 Sep. 2026.
- [2] Bangladesh Public Procurement Authority, “Electronic government procurement (e-GP) system,” <https://www.eprocure.gov.bd/>, accessed: 29 Sep. 2026.
- [3] —, “e-GP: About the system,” <https://bppa.gov.bd/e-gp.html>, accessed: 29 Sep. 2026.
- [4] The Financial Express, “e-GP promises a complete procurement system,” <https://thefinancialexpress.com.bd/views/reviews/e-gp-promises-a-complete-procurement-system>, accessed: 29 Sep. 2026.
- [5] Government of the People’s Republic of Bangladesh, “The public procurement act, 2006 (act no. 24 of 2006),” 2006.
- [6] —, “The public procurement rules, 2008,” 2008.
- [7] S. Lewis-Faupel, Y. Neggers, B. A. Olken, and R. Pande, “Can electronic procurement improve infrastructure provision? Evidence from public works in India and Indonesia,” *American Economic Journal: Economic Policy*, vol. 8, no. 3, pp. 258–283, 2016.
- [8] A. Neupane, J. Soar, K. Vaidya, and J. Yong, “Willingness to adopt e-procurement to reduce corruption: Results of the PLS path modeling,” *Transforming Government: People, Process and Policy*, vol. 8, no. 4, pp. 500–520, 2014.
- [9] K. Vaidya, A. S. M. Sajeev, and G. Callender, “Critical factors that influence e-procurement implementation success in the public sector,” *Journal of Public Procurement*, vol. 6, no. 1/2, pp. 70–99, 2006.
- [10] United Nations Commission on International Trade Law, “UNCITRAL model law on public procurement,” 2011.
- [11] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

- [12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [13] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.
- [14] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.
- [15] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, “MTEB: Massive text embedding benchmark,” in *Proc. 17th Conf. European Chapter of the Association for Computational Linguistics (EACL)*, 2023, pp. 2014–2037.
- [16] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [17] A. Kane and pgvector contributors, “pgvector: Open-source vector similarity search for Postgres,” <https://github.com/pgvector/pgvector>, accessed: 29 Sep. 2026.
- [18] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [19] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [20] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-bench and chatbot arena,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.
- [21] F. Doshi-Velez and B. Kim, “Towards a rigorous science of interpretable machine learning,” arXiv:1702.08608, 2017.
- [22] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

- [23] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST standard for role-based access control,” *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001.
- [24] S. Haber and W. S. Stornetta, “How to time-stamp a digital document,” *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.
- [25] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, 1999.
- [26] S. A. Crosby and D. S. Wallach, “Efficient data structures for tamper-evident logging,” in *Proc. 18th USENIX Security Symposium*, 2009, pp. 317–334.
- [27] M. Dworkin, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” National Institute of Standards and Technology, Tech. Rep. SP 800-38D, 2007.
- [28] “BDTender: Tender notices in Bangladesh,” <https://bdtender.com/>, accessed: 29 Sep. 2026.
- [29] “TenderBazar,” <https://tenderbazar.com/>, accessed: 29 Sep. 2026.
- [30] “AllTender.com,” <https://alltender.com/>, accessed: 29 Sep. 2026.
- [31] “TendersInfo: Global tenders,” <https://www.tendersinfo.com/>, accessed: 29 Sep. 2026.
- [32] Government of the People’s Republic of Bangladesh, “Information and communication technology act, 2006 (amended 2013),” 2006.
- [33] D. Clegg and R. Barker, “Case method fast-track: A RAD approach,” 1994.
- [34] European Parliament and Council, “Regulation (EU) 2016/679 (general data protection regulation),” 2016.
- [35] OWASP Foundation, “OWASP top 10:2021,” <https://owasp.org/Top10/>, 2021.
- [36] —, “Application security verification standard 4.0.3,” <https://owasp.org/www-project-application-security-verification-standard/>, 2021.
- [37] P. A. Grassi *et al.*, “Digital identity guidelines: Authentication and lifecycle management,” National Institute of Standards and Technology, Tech. Rep. SP 800-63B, 2017.
- [38] National Institute of Standards and Technology, “Secure hash standard (SHS),” NIST, Tech. Rep. FIPS PUB 180-4, 2015.
- [39] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” RFC 7519, Internet Engineering Task Force, 2015.

- [40] PCI Security Standards Council, “Payment card industry data security standard, v4.0,” 2022.
- [41] World Wide Web Consortium, “Web content accessibility guidelines (WCAG) 2.1,” W3C Recommendation, <https://www.w3.org/TR/WCAG21/>, 2018.
- [42] ISO/IEC, “ISO/IEC 25010:2011 systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – system and software quality models,” 2011.
- [43] ———, “ISO/IEC 27001:2022 information security, cybersecurity and privacy protection – information security management systems – requirements,” 2022.
- [44] Association for Computing Machinery, “ACM code of ethics and professional conduct,” <https://www.acm.org/code-of-ethics>, 2018.
- [45] IEEE, “IEEE code of ethics,” <https://www.ieee.org/about/corporate/governance/p7-8.html>, 2020.

Appendix A

Functional Requirements with Acceptance Criteria

The following is the requirement baseline of 24 February 2026 as agreed by the team.

Requirement	Description and acceptance criterion
FR-01 General Information Access (M)	Unregistered users can view background, policies, legal notices, news, events, help/support, about-us and contact pages, and an explanation of how the platform works. <i>Acceptance:</i> all public pages are accessible without login and linked from the homepage.
FR-02 Browse Active Public E-Tenders (M)	Unregistered users see only buyer name, title, category, publication and deadline dates, and a short summary. Descriptions, financial terms, eligibility documents and files require registration. <i>Acceptance:</i> public users can view summaries but not detailed documentation.
FR-03 Advanced Search (Semantic) (M)	Semantic matching with filters (category, location, deadline, budget, organisation, visibility). <i>Acceptance:</i> relevant results returned; filters work.
FR-04 User Registration (M)	Two user types: master account holder (organisation admin) and normal member. An organisation may be buyer, seller or both. The master provides e-mail and mobile OTP verification, NID, trade licence, TIN and VAT certificates; members provide e-mail/OTP and NID and gain access once approved by the master. A system administrator verifies NIDs manually. <i>Acceptance:</i> registration completes only after verification and document submission.
FR-05 Authentication (M)	Login, logout, e-mail password reset, optional 2FA, OTP for sensitive actions. <i>Acceptance:</i> login, reset and OTP flows work.
FR-06 Profile Management (M)	One master per company; the master invites users and assigns roles; users accept or decline; users may request affiliation with an organisation's unique code. <i>Acceptance:</i> verification status visible.
FR-07 Notifications & Messaging (M)	Tender-wise inter-company group chat with at least two members per side including their owners, admin-managed membership, and no one-to-one cross-company chat. Intra-company one-to-one chat and broadcast groups. Notifications for user, organisation and workflow events. <i>Acceptance:</i> notifications generated; message history persists.

Requirement	Description and acceptance criterion
FR-08 Tender Creation (M)	Create, edit, cancel, schedule and publish tenders with title, budget, deadline, category, visibility, required documents and bid-bond amount; single-item or packaged lots. <i>Acceptance:</i> lifecycle operates correctly.
FR-09 Vendor Matching (M)	Recommend vendors by profile, certifications, past performance and relevance. <i>Acceptance:</i> ranked recommendations with explanations.
FR-10 Bid Comparison (M)	Side-by-side comparison of pricing and documents. <i>Acceptance:</i> comparison view shows all required information.
FR-11 Notice of Award (M)	Buyer issues NOA; seller accepts online (consuming credits); buyer notified; unselected sellers refunded their bid bond. <i>Acceptance:</i> NOA recorded with an audit trail.
FR-12 Restricted Invitations (S)	Exclusive tenders visible only to enlisted vendors. <i>Acceptance:</i> non-invited vendors cannot view them.
FR-13 Contract Draft & Completion (M)	Draft contract template; physical signing; both parties confirm in the system, which ends the platform's involvement. <i>Acceptance:</i> confirmation recorded.
FR-14 Seller Reviews (S)	Buyer reviews seller after completion; stored on the seller's profile. <i>Acceptance:</i> reviews saved and linked.
FR-15 Tender Feed (M)	Personalised vendor dashboard. <i>Acceptance:</i> relevant tenders appear based on the profile.
FR-16 Bid Submission (M)	Upload technical and financial proposals and the buyer-required documents; pay the bid bond. <i>Acceptance:</i> buyer can reject bids lacking required documents.
FR-17 Bid Status (M)	Submitted, under evaluation, awarded, rejected. <i>Acceptance:</i> status changes timestamped.
FR-18 Credit Points (M)	Purchase and manage credits. <i>Acceptance:</i> payment updates balance and history.
FR-19 Bid-Bond Handling (M)	Hold bid bonds; refund unsuccessful bidders automatically. <i>Acceptance:</i> transactions recorded and refundable.
FR-20 Publishing Cost Configuration (M)	Global default pricing; admin discounts for specific organisations. <i>Acceptance:</i> configured pricing applied at publishing.
FR-21 Admin Dashboard (M)	Manage users, verify documents, moderate, block/unblock, configure costs. <i>Acceptance:</i> admin actions logged.
FR-22 Audit Trail (M)	All critical actions logged with timestamp and identity. <i>Acceptance:</i> logs retrievable and tamper-evident.
FR-23 Disputes (C)	Users raise disputes; admins track resolution.
FR-24 Reporting (C)	Tender volume, vendor performance, usage and revenue reports; exportable.
FR-25 Payment Gateway (M)	At least one secure gateway. <i>Acceptance:</i> sandbox and production transactions succeed.

Requirement	Description and acceptance criterion
FR-26 External Verification APIs (C)	Optional third-party identity/financial verification.

Appendix B

Selected API Endpoints

The back end exposes 99 endpoints; representative ones are listed below.

Method	Path	Purpose
POST	/api/auth/login	Authenticate; set tokens
POST	/api/auth/password-reset/request	E-mail a single-use reset link
POST	/api/org/orgs	Register organisation with KYC documents
POST	/api/org/invitations	Invite a member by e-mail
PATCH	/api/org/members/{id}/role	Change a member's role
POST	/api/org/enlist/{org_id}	Enlist a vendor for restricted tenders
POST	/api/auth/admin/verify/{org_id}	Verify a pending master account
POST	/api/auth/admin/pricing	Set token price and publish/bid costs
POST	/tenders/buyer/create-from-pdf	Queue e-GP PDF import (returns task ID)
POST	/tenders/buyer/publish-with-documents	Publish a tender (debits tokens)
GET	/tenders/public/active	Public tender summaries
GET	/tenders/{id}/recommendations	Explainable vendor recommendations
GET	/search/tenders?q=	Hybrid semantic search
POST	/bids/vendor/submit-with-documents	Submit bid with documents
GET	/bids/buyer/tender/{id}/compare	Side-by-side comparison
POST	/bids/buyer/tender/{id}/evaluate	Start AI evaluation run
POST	/bids/buyer/{bid_id}/accept	Accept bid and create award
POST	/contracts/{id}/reviews	Submit mutual review
POST	/payments/purchase	Buy tokens
POST	/api/messages/threads/group	Create group or tender room
GET	/admin/audit/verify	Verify the audit hash chain